

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Grafové databáze – přehled a srovnání



2024

Vedoucí práce:
Mgr. Tomáš Urbanec

Jan Gaja

Studijní program: Informatika,
Specializace: Obecná informatika

Bibliografické údaje

Autor: Jan Gaja
Název práce: Grafové databáze – přehled a srovnání
Typ práce: bakalářská práce
Pracoviště: Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby: 2024
Studijní program: Informatika, Specializace: Obecná informatika
Vedoucí práce: Mgr. Tomáš Urbanec
Počet stran: 54
Přílohy: elektronická data v úložišti katedry informatiky
Jazyk práce: český

Bibliographic info

Author: Jan Gaja
Title: Graph databases – overview and comparison
Thesis type: bachelor thesis
Department: Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense: 2024
Study program: Computer Science, Specialization: General Computer Science
Supervisor: Mgr. Tomáš Urbanec
Page count: 54
Supplements: electronic data in the storage of department of computer science
Thesis language: Czech

Anotace

Práce má dva cíle: 1. nastudovat principy ukládání a práce s daty v grafových databázích a získané poznatky sepsat do textu, který čtenáři poskytne dobrou představu o fungování, výhodách a nevýhodách tohoto přístupu k datům. 2. vybrat několik implementací grafových databází, nachystat ukázky jejich použití a pokusit se navrhnout experimenty ukazující silné a slabé stránky zvolených implementací.

Synopsis

Thesis has two goals: 1. study the principles of storing and working with data in graph databases and and write down the knowledge gained in a text that will give the reader a good idea of how it works, advantages and disadvantages of this approach to data. 2. select several implementations of graph databases, prepare examples of their use and try to devise experiments showing the strengths and weaknesses of the selected implementation.

Klíčová slova: grafové databáze; implementace grafových databází

Keywords: graph databases; implementation of graph databases

Děkuji, děkuji, děkuji.

Místopřísežně prohlašuji, že jsem celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

datum odevzdání práce

podpis autora

Obsah

1	Základní pojmy z teorie grafů	9
2	Grafové databáze	12
2.1	Nativní ukládání a zpracování dat	12
2.2	Grafové modely	12
2.2.1	Grafy s vlastnostmi	13
2.2.2	RDF	13
2.2.3	Hypergrafy	13
2.3	Systémy OLTP a OLAP	13
2.4	Výhody grafových databází	15
2.5	Nevýhody grafových databází	16
2.6	Implementace grafových databází	17
2.6.1	Neo4j	18
2.6.2	OrientDB	18
2.6.3	ArangoDB	19
2.7	Dotazovací jazyky v grafových databázích	20
2.7.1	Cypher	20
2.7.2	Gremlin	21
2.7.3	AQL	21
3	Modelování grafových dat	22
3.1	Definice problému	23
3.1.1	Co od aplikace očekáváme?	25
3.1.2	S jakými informacemi aplikace pracuje?	25
3.1.3	Jak spolu tyto informace vzájemně souvisí?	26
3.1.4	Na jaké otázky potřebujeme v aplikaci najít odpověď?	26
3.2	Konceptuální datový model	27
3.2.1	Entity	27
3.2.2	Vztahy	27
3.2.3	Konceptuální model	28
3.3	Logický datový model	29
3.3.1	Vrcholové labely	29
3.3.2	Hranové labely	30
3.3.3	Vlastnosti	32
3.4	Testování modelu	33
3.4.1	Unit testy	33
3.4.2	Profile a Explain	34
3.4.3	Indexy	34
3.4.4	Denormalizace	35
4	Known walk	35
4.1	Příklad použití	36

5	Hledání cest mezi 2 vrcholy	36
5.1	Procházení grafu	37
5.2	Cesta v neohodnoceném grafu	38
5.3	Cesta v ohodnoceném grafu	40
6	Algoritmy	42
6.1	Algoritmy pro hledání cesty v grafu	43
6.1.1	Dijkstra	43
6.1.2	A*	43
6.1.3	Minimální kostra grafu	44
6.1.4	Random walk	45
6.2	Algoritmy pro míru centrality	46
6.2.1	Degree Centrality	46
6.2.2	Closeness Centrality	46
6.2.3	Betweenness Centrality	46
6.2.4	PageRank	47
6.2.5	Použití algoritmů pro míru centrality	47
6.3	Detekce komunit	48
6.3.1	Počítání trojúhelníků	48
6.3.2	Hledání komponent	48
6.3.3	Algoritmus Louvain	49
7	Experimenty	50
	Závěr	51
	Conclusions	52
	Literatura	53

Seznam obrázků

1	Knowledge graph	14
2	Hypergraf	15
3	Rozhodovací strom pro vhodnost použití grafové databáze	24
4	Konceptuální datový model pro sociální síť pro cestovní nadšence.	28
5	Label vrcholu definuje, do které kategorie vrchol spadá.	30
6	Ukázka více labelů pro jeden vrchol.	31
7	Logický datový model pro sociální síť pro cestovní nadšence. . . .	33
8	Graf vytvořený pomocí zdrojového kódu 2.	35
9	Model grafové databáze pro ukázkové příklady v této kapitole. . .	37
10	Průchod algoritmu BFS grafem.	38
11	Průchod algoritmu DFS grafem.	39
12	Podgraf reprezentující výsledek dotazu ve zdrojovém kódu 4. . . .	41
13	Minimální kostra grafu.	44
14	Příklad grafu, na kterém jsou prováděny algoritmy.	47

Seznam tabulek

1	Tabulka souborů pro aplikaci pro cestovní nadšence.	25
2	Tabulka souborů pro ukázkou hledání cest.	37
3	Výsledná tabulka pro dotaz ve zdrojovém kódu 4	40
4	Výsledná tabulka pro dotaz ve zdrojovém kódu 5.	41
5	Random walk	45
6	Výsledky měření centrality	47
7	Počet trojúhelníků a shlukovací koeficient	48
8	Algoritmus Louvain	50

Seznam zdrojových kódů

1	Připojení se k databázi OrientDB v Gremlin Console	22
2	Unit test	34
3	Known walk	36
4	Cesty v neohodnoceném grafu	40
5	Hledání nejkratší cesty v ohodnoceném grafu	42

Úvod

Grafové databáze, stejně jako jiné databáze (například relační, NoSQL) představují systém pro uchovávání a zpracovávání dat. Jsou, jak již název napovídá, postaveny na jednom z oborů diskrétní matematiky jménem teorie grafů, podobně jako jsou relační databáze postaveny na pojmu relace. Pracují tedy s daty v podobě grafu. Díky této struktuře mají skvělé vlastnosti pro aplikace, které ve větší míře pracují se vztahy mezi jednotlivými entitami.

První kapitola obsahuje základní pojmy a definice z teorie grafů používané v textu.

Druhá kapitola se věnuje grafovým databázím. Je představeno několik rozdělení grafových databází. Jsou uvedeny jejich výhody a nevýhody. Ukázány různé implementace grafových databází na trhu a dotazovací jazyky pro ně. Převážně pak je pozornost věnována databázím Neo4j, ArangoDB a OrientDB a jim příslušným dotazovacím jazykům Cypher, AQL a Gremlin.

Třetí kapitola popisuje modelování dat v grafových databázích. Na konkrétním příkladu je ukázán postup modelování.

Čtvrtá kapitola pojednává o předdefinovaných cestách, které jsou známy a specifikovány v rámci databázového schématu.

Pátá kapitola se zabývá hledáním cest mezi dvěma vrcholy.

V šesté kapitole je nahlédnuto na některé z algoritmů využívaných v grafových databázích. Jsou zde přehledově představeny algoritmy pro hledání cest v grafu, algoritmy pro míru centrality a podívali jsme se na problém detekce komunit.

V sedmé kapitole jsou shrnuty výsledky porovnávání třech zvolených grafových databází.

1 Základní pojmy z teorie grafů

V této části jsou uvedeny základní definice a pojmy z teorie grafů používané v textu. Tato kapitola byla napsána pomocí [1], [2], [3] a [4].

Definice 1 (Graf)

Graf G je uspořádaná dvojice $\langle V, E \rangle$, kde V označuje neprázdnou množinu vrcholů a E označuje množinu hran. Každá hrana je určena dvěma vrcholy z V , které spojuje.

V grafu $G = \langle V, E \rangle$ se V nazývá *množina vrcholů* grafu G a označuje se $V(G)$. Podobně E se nazývá *množina hran* grafu G a označuje se $E(G)$.

POZNÁMKA 2 (VRCHOL)

V teorii grafů se pojem vrchol někdy zaměňuje s pojmem uzel. V této práci však budeme používat pojem vrchol.

POZNÁMKA 3 (NEKONEČNÉ GRAFY)

Grafy mohou mít nekonečnou množinu vrcholů. Tyto grafy se nazývají nekonečné. Nekonečné grafy však obecně nelze uchovávat v počítači. V této práci se omezíme na grafy s konečnou množinou vrcholů.

Grafy se dělí na *orientované* a *neorientované*. Hrana v neorientovaném grafu spojující vrcholy u a v se nazývá *neorientovaná* a zapisuje se pomocí množiny obsahující dvojici vrcholů, tedy $\{u, v\}$. Hrana v orientovaném grafu začínající ve vrcholu u a končící ve vrcholu v se nazývá *orientovaná* a pro zápis se používá uspořádaná dvojice, tedy $\langle u, v \rangle$. O hraně začínající i končící ve stejném vrcholu mluvíme jako o *smyčce*. Pro hrany používáme označení *vícenásobné*, pokud existuje mezi dvěma vrcholy hran více. Grafu obsahujícímu vícenásobné hrany se říká *multigraf*.

Grafy se nejčastěji znázorňují pomocí kroužků, které představují vrcholy, a úseček mezi nimi, sloužící pro reprezentaci hran. Pokud se jedná o orientovaný graf místo úseček používáme šipky.

Definice 4 (Podgraf)

Podgraf grafu $G = \langle V, E \rangle$ je graf $H = \langle V_1, E_1 \rangle$, kde $V_1 \subseteq V$ a $E_1 \subseteq E$, tak že každá hrana z E_1 má oba vrcholy ve V_1 .

Neformálně podgraf vznikne z původního grafu odstraněním některých hran, nebo vrcholů spolu s jejich incidentními hranami. Podgraf $H = \langle V_1, E_1 \rangle$ nazveme *indukovaný*, pokud E_1 obsahuje všechny hrany z E , pro které platí, že mají oba konce v množině V_1 .

Definice 5 (Symetrizace orientovaného grafu)

Symetrizace orientovaného grafu $G = \langle V, E \rangle$ je neorientovaný graf $G' = \langle V, E' \rangle$, pokud pro každou hranu $\{u, v\} \in E'$ platí, že $\langle u, v \rangle \in E$, nebo $\langle v, u \rangle \in E$.

Symetrizaci grafu lze popsat jako proces, ve kterém zapomeneme orientaci hran, odstraníme smyčky a vícenásobné hrany nahradíme jednoduchými a vznikne nám tak neorientovaný graf.

Definice 6 (Ohodnocení hran)

Nechť $G = \langle V, E \rangle$ je graf. Funkce $w : E \rightarrow \mathbb{R}$ se nazývá *ohodnocení hran grafu* G . *Ohodnoceným grafem* nazveme trojici $G = \langle V, E, w \rangle$.

Hodnotu $w(e)$ pro hranu $e \in E$ ohodnoceného grafu $G = \langle V, E, w \rangle$ nazýváme *váha* nebo *ohodnocení hrany* e . Obdobně můžeme definovat *ohodnocení vrcholů* jako funkci $w : V \rightarrow \mathbb{R}$.

POZNÁMKA 7 (OHODNOCENÝ GRAF)

Ohodnoceným grafem v textu budeme vždy myslet graf $G = \langle V, E, w \rangle$, kde přiřazení w je nezáporné ohodnocení hran.

Definice 8 (Sousedství)

Řekneme, že vrcholy $u, v \in V$ v grafu $G = \langle V, E \rangle$ spolu *sousedí*, pokud existuje mezi nimi hrana $e \in E$. Říkáme, že vrcholy u, v jsou *incidentní* s e . *Sousedství* vrcholu $v \in V$ je množina vrcholů $N(v)$ obsahující všechny vrcholy, se kterými vrchol v sousedí.

Definice 9 (Stupeň vrcholu)

Stupněm vrcholu $u \in V$ v grafu $G = \langle V, E \rangle$ se rozumí počet hran s ním incidentních. Značíme jej $\deg(u)$.

Vrcholy se stupněm 1 se v teorii grafů nazývají *koncové* vrcholy, nebo *listy*. Vrcholy se stupněm 0 se nazývají *izolované*. Stupeň vrcholu u lze v orientovaném grafu dále rozdělit podle hran vcházejících a vycházejících na *vstupní stupeň vrcholu* u definovaný jako $\deg_{in}(u) = |\{\langle v, u \rangle | v \in V\}|$ a *výstupní stupeň vrcholu* u definovaný jako $\deg_{out}(u) = |\{\langle u, v \rangle | v \in V\}|$.

Definice 10 (Sled)

Sled v grafu $G = \langle V, E \rangle$ je sekvence vrcholů a hran tvaru $v_0, e_1, v_1, e_2, \dots, e_n, v_n$, kde vrchol $v_i \in V$ pro všechna $i \in \{0, 1, 2, \dots, n\}$ a pro hranu e_i platí:

- $e_i = \{v_{i-1}, v_i\} \in E$ pro všechna $i \in \{1, 2, \dots, n\}$, je-li G neorientovaný.
- $e_i = \langle v_{i-1}, v_i \rangle \in E$ pro všechna $i \in \{1, 2, \dots, n\}$, je-li G orientovaný.

Definice 11 (Cesta)

Cesta je sled, kde se vrcholy ani hrany neopakují.

V neohodnocených grafech má cesta obsahující $n + 1$ vrcholů délku n . Délka cesty v ohodnoceném grafu je rovna součtu ohodnocení hran na cestě.

Definice 12 (Kružnice)

Kružnice je cesta, kde počáteční a koncový vrchol je stejný. U orientovaného grafu používáme pojem *cyklus*.

O grafu $G = \langle V, E \rangle$ řekneme, že je *cyklický*, pokud se v grafu vyskytují kružnice, nebo cykly. Pokud v grafu neexistuje kružnice, graf nazveme *acyklický*.

Definice 13 (Vzdálenost)

Pro vrcholy u, v grafu G definujeme *vzdálenost* vrcholu u a vrcholu v definujeme jako délku cesty z u do v , která je ze všech délek cest z u do v nejmenší. Značíme ji $dist(u, v)$.

Definice 14 (Souvislý graf)

Neorientovaný graf $G = \langle V, E \rangle$ nazýváme *souvislý* pokud pro každé dva vrcholy $u, v \in V$ platí, že existuje cesta z u do v . U orientovaného grafu $H = \langle V_H, E_H \rangle$ rozlišujeme slabou a silnou souvislost. Graf H je *slabě souvislý*, pokud jeho symetrizace je souvislý graf. O grafu H řekneme, že je *silně souvislý*, pokud pro každé dva vrcholy $u, v \in V_H$ platí, že existuje cesta z u do v .

Komponenta grafu G je každý jeho maximální souvislý podgraf.

Definice 15 (Strom)

Strom je souvislý graf $G = \langle V, E \rangle$ neobsahující kružnice.

Množina navzájem nepropojených stromů se nazývá *les*.

Definice 16 (Kořenový strom)

Kořenový strom je dvojice $T = \langle G, r \rangle$, kde $G = \langle V, E \rangle$ je strom a $r \in V$. Vrchol r nazýváme *kořen*.

Opomeňme ohodnocení hran v grafu. *Hloubka vrcholu* v ve stromě $T = \langle G, r \rangle$ je rovna $dist(r, v)$. Speciálně hloubka kořene r je 0. *Výškou kořenového stromu* T rozumíme nejdelší vzdálenost z kořene do některého z vrcholů v T .

Vrchol v nazýváme *předchůdce* (rodič) vrcholu u , pokud v kořenovém stromu T existuje cesta z kořene r do u ve tvaru $r \dots, v, e, u$. Vrchol u se pak nazývá *následník* (přímý potomek) v . Pokud vrchol v se vyskytuje na cestě z kořene do u , nazýváme vrchol v *předek* u a vrchol u *potomek* v . Vrcholy mající stejného rodiče označujeme jako *sourozence*. *Listem* nazýváme vrchol, který v kořenovém stromu nemá žádné následníky. Naopak *vnitřní vrchol* je nazýván ten vrchol, který není listem.

2 Grafové databáze

Grafová databáze je specializovaná NoSQL databáze pro uchovávání a dotazování dat, které jsou mezi sebou provázány pomocí vztahů. Jedná se o systém, který rozšiřuje grafový model o operace CREATE, READ, UPDATE a DELETE (zkráceně CRUD). Data v databázi jsou uchovávána ve formě vrcholů a hran, které spolu tvoří graf. Cílem této části je představit základní vlastnosti grafových databází, jejich výhody v porovnání s jinými typy databází a na závěr je nahlédnuto na některé implementace. Při psaní jsem čerpal z [5], [6] a [11].

2.1 Nativní ukládání a zpracování dat

[7] Grafové databáze můžeme mezi sebou rozlišovat podle zpracování a ukládání dat.

Grafová databáze má *nativní grafové uložště*, pokud umožňuje uchovávání a správu dat v grafové podobě. Tento přístup zefektivňuje následnou práci s daty. Nicméně ne všechny grafové databáze se vydávají při ukládání dat stejnou cestou. Některé z nich tato data serializují do jiných typů databází. Tento způsob se především opírá o vyspělost a známost jiných technologií, hlavně co se týče relačních databází.

Druhou vlastností je *index-free adjacency*. Ta nám říká, že v uložšti jsou s každým vrcholem mimo jiné uloženy přímé odkazy (pointery) na sousední vrcholy. Pro přístup k sousedům každého vrcholu tedy není potřeba žádná další datová struktura, nebo globální index. Obdobně to platí pro hrany. S každou hranou máme též uloženy ukazatele na koncové vrcholy, které jsou s danou hranou incidentní. Výsledkem je skvělý výkon při procházení grafem, protože z aktuálního vrcholu se lze dostat na specifický vrchol s konstantní časovou složitostí. Navíc výkon není závislý na velikosti grafu v databázi, nýbrž na množství procházených dat. Toto je obrovská výhoda ve srovnání s relačními databázemi, kde je zpracování požadavku závislé na velikosti spojovaných tabulek. Nevýhodou tohoto přístupu je, že některé dotazy mohou být na výkon nebo paměť mnohem náročnější. Zejména ty, při kterých nedochází k průchodu grafem.

Nativními grafovými databázemi označujeme ty, které s daty manipulují a uchovávají je v grafové podobě. Pracují efektivněji při průchodu grafem. Pro práci s velkým množstvím hustě propojených dat je tato architektura navrhnutá pro zacházení s daty nezbytná. Nenativní grafové databáze jsou bezpečnější volbou pro malé projekty a ne příliš propojená data, neboť obvykle staví na technologiích, které jsou známé. V této práci se budeme dále zabývat pouze nativními grafovými databázemi.

2.2 Grafové modely

Grafové databáze lze klasifikovat podle různých grafových modelů, jako jsou například grafy s vlastnostmi (anglicky property graphs), RDF (Resource Description Framework) a hypergrafy.

2.2.1 Grafy s vlastnostmi

Jak z názvu vyplývá, jedná se o model, ve kterém jsou grafy doplněny o vlastnosti. Tyto vlastnosti mohou být přiřazeny jak vrcholům, tak hranám. Vlastnosti jsou reprezentované pomocí dvojic klíč-hodnota. Vrcholům jsou též přiřazeny labely. Pomocí nich lze vrcholy seskupovat. Hraný jsou pojmenované a orientované. U každé hrany tedy jde určit počáteční a koncový vrchol. Jedná se o koncept, který je nejvíce využíván v grafových databázích.

2.2.2 RDF

RDF model umožňuje vyjadřovat tvrzení o věcech v podobě jednoduchých vět, které gramaticky sestávají z podmětu, přísudku a předmětu. Tyto věty nazýváme tvrzení. Tvrzení samozřejmě nevyjadřujeme jako věty v přirozeném jazyku, ale v počítačové notaci. RDF model zavádí následující strukturu zvanou trojice: *subjekt predikát objekt*. Subjekt je obdobou podmětu, predikát je obdobou přísudku a objekt je obdobou předmětu. Každá věta popisuje hranu spojující vrcholy definované subjektem a objektem. Predikát je pak pojmenováním hrany. RDF model je známý ze sémantického webu, kde je použit pro odvození informací pomocí sémantického označení odkazů spojujících webové zdroje.[8]

Tento model též nachází využití v aplikacích využívajících znalostní grafy (anglicky knowledge graphs). Příkladem těchto grafů je například Google Knowledge Graph ¹. Google jej využívá například ve svém hlasovém ovládání Google Asistenta, nebo od května 2012 data získaná z tohoto grafu vizualizuje v tzv. Knowledge Graph panelu. [9] Knowledge graph pro nizozemského informatika Edsgera Dijkstra zobrazuje obrázek 1.

2.2.3 Hypergrafy

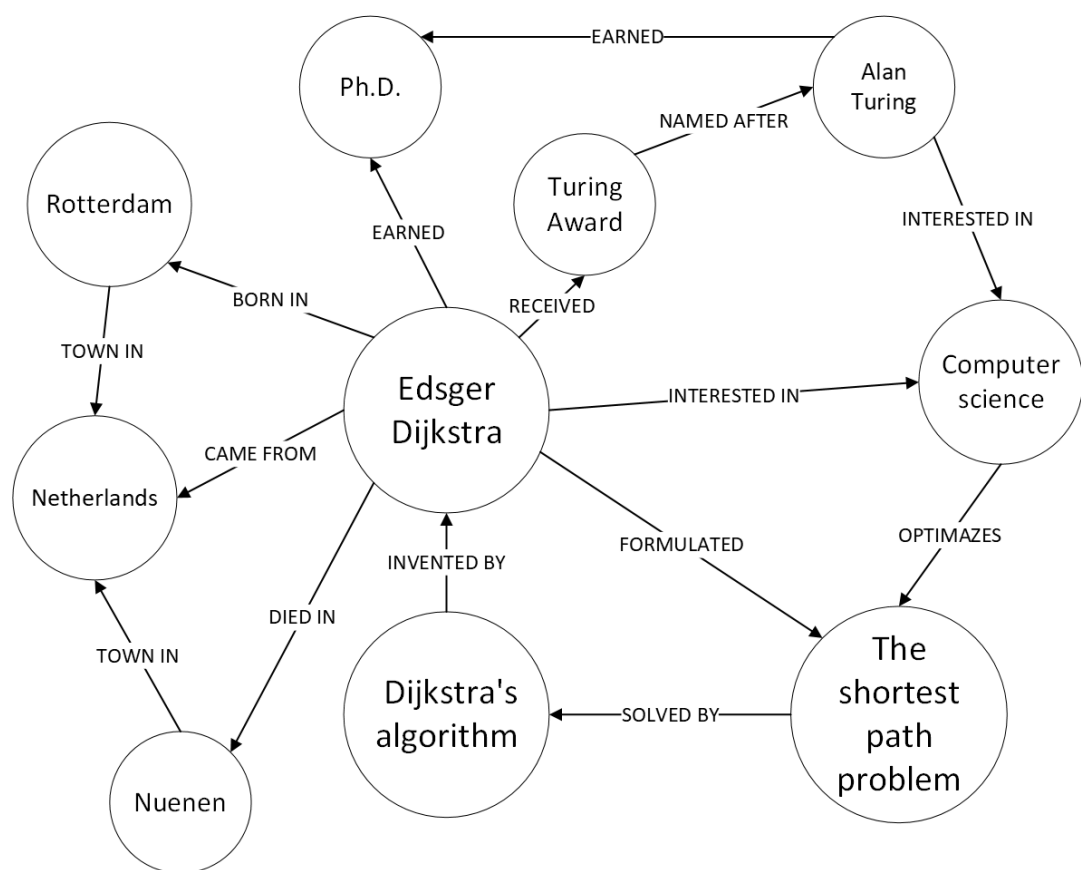
Hypergraf vznikl zobecněním klasického grafového modelu. Hypergraf obsahuje speciální hrany zvané hyperhrany, které mohou spojit více vrcholů. Narozdíl od klasické orientované hrany, hyperhrana může mít libovolné množství počátečních a koncových vrcholů. Hypergrafy mohou být užitečné v aplikacích, které zacházejí s vazbami o kardinalitě M:N. Vždy je možné reprezentovat informace v hypergrafu jako property graph. I když s použitím více vztahů a zprostředkujících vrcholů. Příklad hypergrafu je ukázán na obrázku 2.

2.3 Systémy OLTP a OLAP

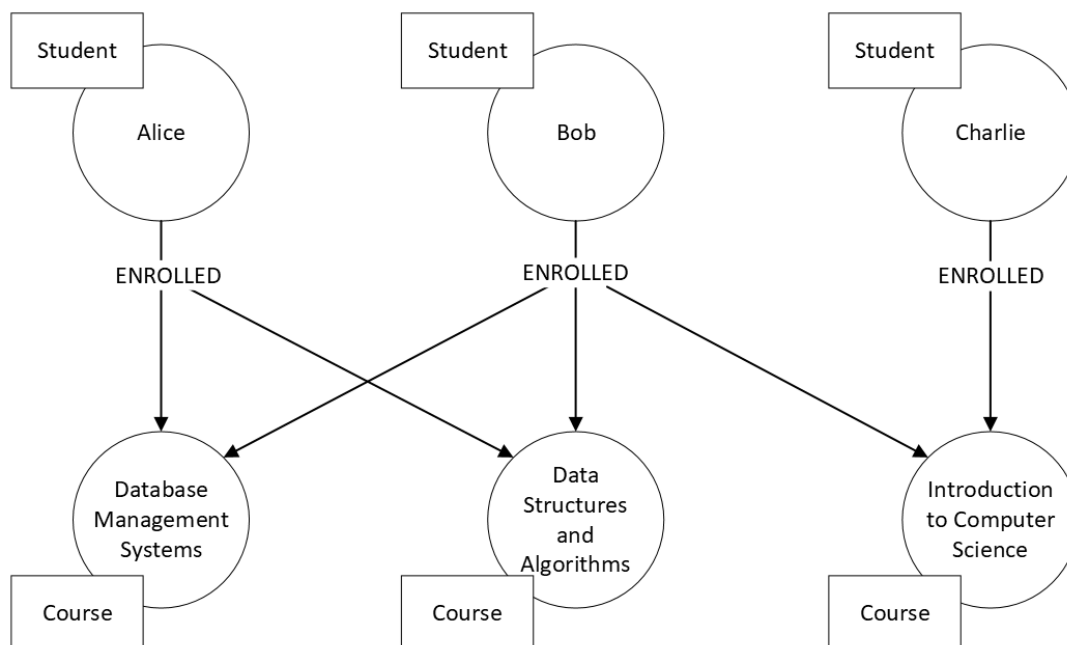
[10] Grafové databáze dále můžeme rozdělit do dvou kategorií: *OLAP* (On-Line Analytical Processing) a *OLTP* (On-Line Transactional Processing).

- OLTP systémy jsou navrženy pro rychlé a efektivní zpracování velkého množství transakcí. Nejsou vhodné pro analýzu dat. Grafové databáze jsou

¹<https://blog.google/products/search/introducing-knowledge-graph-things-not/>



Obrázek 1: Knowledge graph.



Obrázek 2: Hypergraf.

obvykle navrhnuty pro použití v OLTP systémech. Hodí se pro aplikace, ve kterých jsou data v reálném čase přidávána, mazána a modifikována. Nachází využití například v doporučovacích a bankovních systémech, sociálních sítích a při odhalování podvodů.

- OLAP systémy jsou navrženy pro komplexní analýzu velkého objemu dat. Data se z databáze ve většině případů čtou. Zpracovávají malé množství časově a výpočetně náročných dotazů. Využívány jsou například v analýze znalostních grafů.

2.4 Výhody grafových databází

Nativní grafové databáze uchovávají jak samotné entity, tak všechny vztahy mezi entitami. Při dotazování nedochází k výpočtu těchto vztahů, jak tomu je například u relačních databází pomocí klauzule JOIN. To zrychluje zpracování některých typů dotazů. Množství dat obecně výkon nezhoršuje, neboť čas pro vyhodnocení dotazu závisí pouze na velikosti podgrafu, kterým procházíme. Pro aplikace, které pracují se vztahy mezi entitami ve velké míře, je proto nativní grafová databáze ideálním řešením.

Pomocí vrcholů a hran lze často lépe vystihnout dané problémy, než například pomocí tabulek. Díky grafové struktuře můžeme data ukládat a manipulovat s nimi přirozeným způsobem. Popisují jak nad daty v rámci aplikace přemýšlíme. Lze tak dosáhnout datových modelů, které jsou srozumitelné i laikům, což ve

velkých projektech, na kterých pracuje velká spousta lidí, může být obrovská výhoda. Navíc ve spoustě případů je dotazování prostřednictvím grafových dotazovacích jazyků přehlednější než dotazy v jiných databázích. To platí především pro dotazy pracující se vztahy mezi entitami.

Grafové databáze obecně nevynucují definování schématu. Často se označují jako schema-less. Tedy struktura dat se může měnit v závislosti na potřebách aplikace. Přidávání nových vlastností, vrcholů, hran a labelů nenarušuje chod aplikace. Což přináší pozitiva pro vývojáře a budoucí vývoj. Ovšem základní model dat by měl být znám, abychom mohli s daty pracovat.

Dotazy v grafových databázích mohou vracet různé typy dat, aniž by ztrácely výsledky smysl. V relačních databázích lze v jednom dotazu vracet výsledky pro různé entity pomocí klauzule UNION, ale vrácená data potrádají smysl, neboť vrácená tabulka obsahuje tato data různých entit smíchaná dohromady. V grafových databázích toho lze využít například k získání všech dostupných informací ke konkrétnímu zákazníkovi, které máme v databázi, v rámci jednoho dotazu.

Grafové databáze lze využít k uchovávání vztahů mezi slovy a frázemi. Tedy v kombinaci s počítačovým zpracováním jazyka jsou vhodné v oborech jako data science, nebo machine learning.

I přes problematické rozdělení grafových dat (sharding) mnoho implementací grafových databází nabízí horizontální škálování. To umožňuje rozdělit data mezi více výpočetních strojů. To je klíčové při uchování obrovských grafů a pro snížení zátěže jednotlivých strojů. Je spousta technik jak data rozdělit. Například pomocí hashování unifikátorů, nebo pomocí grafových vzorů. Ale volba správné techniky je obtížná.

Každý vrchol a každá hrana v grafu mají své vlastní jedinečné identifikátory (ID). To je rozdíl oproti relačním databázím, ve kterých sloupec s identifikátory může a nemusí být v tabulce přítomný. Tyto identifikátory mohou být uživatelsky měnitelné v některých grafových uložiscích, ale častějším případem jsou generovány systémem automaticky.

2.5 Nevýhody grafových databází

Grafové databáze jsou mezi ostatními druhy databází poněkud mladé. Narozdíl od relačních databází je všechny mezi sebou nepropojuje jeden standardizovaný dotazovací jazyk. Jeho absence škodí jejich popularitě, neboť pro vývojáře znamená, že pro práci s více grafovými databázemi je nucen používat více různých dotazovacích jazyků, místo specializování se na jeden univerzální. Navíc při změně grafové databáze hrozí, že bude nutné přepsat všechny dotazy do jiného dotazovacího jazyka. Spousta databází používá speciálně upravený jazyk SQL, umožňující základní práci s databázemi. Jednou z nich je OrientDB. Nejednotnost dotazovacího jazyka by mohla být v blízké době minulostí. Podle dostupných informací se pracuje na normovaném jazyku GQL, což je zkratka pro Graph Query

Language².

Data v podobě grafu nejsou dobrou volbou pro aplikace využívající transakce. Při velkém objemu transakčních dat nejsou tak efektivní jako jiné.

Grafovým databázím dělají problém dotazy, které operují nad celým grafem. Jedná se především o filtrování a agregování. Pokud se v aplikaci soustředíme především na takové typy operací je lepší zvolit jiné databázové technologie.

Jelikož jsou grafové databáze často schema-less může docházet k nekonzistenci ukládaných dat. Ne všechny implementace grafových databází totiž disponují mechanismy, které by v případě potřeby nekonzistenci zabránily. Konzistence dat je tedy často řešena pouze na aplikační úrovni.

2.6 Implementace grafových databází

Existuje spousta implementací grafových databází. Liší se například nativností; dotazovacím jazykem, škálovatelností a podporou ACID transakcí. ACID je zkratka pro čtyři žádané vlastnosti databázových transakcí: atomicita, konzistence, izolovanost a trvalost. Jakou z nich využít pro naši aplikaci je otázka preferencí. My se zaměříme na tři implementace, a to **Neo4j**, **OrientDB** a **ArangoDB**. Dalšími známými databázemi jsou například:

- **Amazon Neptune**³ – rychlý, spolehlivý, plně řízený a vysoce zabezpečený systém pro hustě propojené grafy. Podporuje jak property graph tak RDF. Databáze byla představena v roce 2017 společností Amazon. Výhodou je podpora známých dotazovacích jazyků Gremlin, openCypher a SPARQL.
- **TigerGraph**⁴ – moderní, nativní grafová databáze pro ukládání property graph. Nabízí vestavěné paralelní výpočty nad grafy.
- **AllegroGraph**⁵ – výkonná multi-model open-source databáze. Data mohou být uložena ve formě RDF grafu, vektorů, nebo dokumentů. AllegroGraph je kompatibilní se standardy W3C/ISO a podporuje dotazy prostřednictvím SPARQL a Prologu. Dle [5] nemá nativní grafové zpracování. Předností je kombinování geografických, časových informací a údajů z analýzy sociálních sítí současně v rámci jednoho dotazu. To umožňuje odpověď na otázky jako: Najdi zda nejvlivnější kamarád Alice (využití analýzy sociálních sítí) v poslední dekádě (časový údaj) provedl platbu kartou 100 kilometrů od Olomouce (geografický údaj).
- **FlockDB** – vyvinuta společností Twitter. Jedná se o nenativní open-source grafovou databázi, navrženou pro sociální aplikace.

²<https://www.gqlstandards.org>

³<https://aws.amazon.com/neptune>

⁴<https://www.tigergraph.com>

⁵<https://allegrograph.com/>

- **JanusGraph** ⁶ – jedná se o open-source projekt společnosti The Linux Foundation. Jedná se o odnož grafové databáze Titan. Je založen na Apache TinkerPop a tedy lze klást dotazy prostřednictvím jazyka Gremlin. Kromě online transakčního zpracování (OLTP) podporuje JanusGraph globální grafovou analýzu (OLAP) díky integraci Apache Spark.

2.6.1 Neo4j

[12], [13] Databázový systém Neo4j je nejrozšířenější open-source grafovou databází na trhu s největší uživatelskou základnou. Nabízí vysoký výkon při práci s grafy. Její první verze byla vyvinuta v roce 2002 a vznikla z důvodu nespokojenosti jejich zakladatelů s výkonem relačních databází. Implementována je v Javě. Jedná se o nativní grafovou databázi. Je využívána hromadou světoznámých společností jako je například Microsoft, eBay, nebo LinkedIn. Klást dotazy můžeme prostřednictvím jazyka Cypher.

Neo4j nabízí velké množství způsobů jak interagovat s grafovými daty. Každý z nich je určen pro jinou cílovou skupinu. Například v této práci je využita aplikace Neo4j Desktop, která slouží pro správu lokálních instancí Neo4j. Verze používané aplikace je 1.5.9. Dalším produktem je cloudový databázový servis pojmenovaný Neo4j AuraDB. Vznikl v roce 2019. Je také dostupný v bezplatné verzi, která je však omezená – lze v ní spravovat nejvýše 200 tisíc vrcholů a 400 tisíc hran. Neo4j má též spoustu rozšiřujících knihoven a nástrojů, které lze do produktů doinstalovat (pokud již nejsou součástí). Například se jedná o nástroj pomáhající při migrování dat z relační databáze do Neo4j, standardní knihovna procedur a funkcí apod. V práci je pak ukázána knihovna s užitečnými grafovými algoritmy.

2.6.2 OrientDB

[14] OrientDB je open-source NoSQL databázový systém implementovaný v Javě. Byl představen v roce 2010 (komerčně 2011). Jedná se o pokročilou multi-model databázi - podporuje grafový, dokumentový, klíč-hodnota a objektový model. Manipulovat s databází můžeme pomocí programovacího jazyka Java, speciálně upraveného SQL a dotazovacího jazyka Gremlin. Uživatelům je pak umožněno interagovat s databází skrze Java API, HTTP REST API, konzolu nebo prostřednictvím grafického uživatelského rozhraní – OrientDB Studio. Pokud OrientDB Server běží lokálně, OrientDB Studio lze najít ve webovém prohlížeči na adrese <http://localhost:2480>. OrientDB je schématicky flexibilní. Umožňuje uchovávat schema-full, schema-less a schema-hybrid data.

V databázi OrientDB jsou identifikátory řešeny následovně. Nejmenší jednotkou v databázi je záznam (record). Rozlišujeme 4 typy záznamů: Documents, Blobs, Vertices a Edges. Pro každý záznam pak je automaticky vygenerováno ID záznamu, označeno jako RID (Record ID). RID je uchováván ve formátu *#<cluster>:<position>*.

⁶<https://janusgraph.org>

- <cluster> označuje identifikátor clusteru, do kterého záznam patří.
- <position> udává absolutní pozici záznamu v rámci clusteru.

V práci je využívána bezplatná verze OrientDB Community Edition ve verzi 3.2.23. OrientDB nabízí i placenou verzi Enterprise Edition, která poskytuje výhody jako správu zálohování, migrační nástroje, monitorování systému a CRUD operací v reálném čase a jiné.

2.6.3 ArangoDB

[15] ArangoDB stejně jako předchozí dvě je open-source databáze. Je multi-model databáze. Lze ji použít jako plnohodnotné úložiště dokumentů, grafů a párů klíč-hodnota, full-text vyhledávací engine nebo jakoukoli kombinaci těchto technologií. První verze byla představena společností ArangoDB Inc. v roce 2012. S daty je možno pracovat prostřednictvím jazyka AQL (ArangoDB Query Language). Pokud server provozující ArangoDB běží lokálně je možno interagovat s databází skrze webové rozhraní na adrese 127.0.0.1:8529, nebo prostřednictvím textového rozhraní *arangosh*. Lze též komunikovat prostřednictvím HTTP API, nebo driveru.

Data jsou ukládána ve známém formátu JSON. Nejmenší jednotkou v databázi je *dokument* – jeden záznam dat. Dokumenty jsou dále organizovány do kolekcí. Kolekce mohou obsahovat libovolné množství dokumentů. Tímto způsobem můžeme seskupovat dokumenty. Dokumenty jsou ukládány v binární podobě zvané *VelocityPack*. Každý dokument má dále atributy, které slouží jako vlastnosti. Navíc obsahují 3 povinné atributy, nazývané *systémové*, které začínají podtržítkem. Mají datový typ string a všechny jsou určeny pouze ke čtení.

- *__key* – nazývá se *dokumentový klíč*. Každý dokument v rámci kolekce má hodnotu tohoto atributu unikátní. Dokumentový klíč lze specifikovat při vytváření dokumentu. Pak již nelze změnit. Pokud není specifikován, je atributu přiřazena hodnota automaticky.
- *__id* – nazývá se *dokumentový identifikátor*. Identifikuje dokument v rámci databáze. Je ve formě <název-kolekce>/<klíč-dokumentu>.
- *__rev* – nazývá se *dokumentová revize*. Hodnota atributu je automaticky obdržena serverem.

Hrany v databázi jsou uloženy ve speciálních dokumentech a kolekcích, nazvané *hranové*. A mají další dva systémové atributy.

- *__from* – hodnota atributu je rovna dokumentovému identifikátoru počátečního vrcholu.
- *__to* – hodnota atributu je rovna dokumentovému identifikátoru koncového vrcholu.

Další atributy jsou uživatelsky definovatelné.

Ačkoli je ArangoDB schema-less, umožňuje vynutit strukturu dokumentů v rámci kolekcí. Toho lze dosáhnout pomocí definování JSON Schema⁷ formátu.

ArangoDB nabízí též placenou verzi, nabízející například vyšší výkonnost, zabezpečení a škálování grafů.

2.7 Dotazovací jazyky v grafových databázích

Pomocí dotazovacích jazyků lze formulovat dotazy, které lze klást na databázi. Pomocí nich lze definovat *grafové vzory*. Ty popisují, jakým způsobem grafem procházíme. Na základě procházených vrcholů a hran následně můžeme dostat odpovědi na dotazy. Jak již bylo zmíněno pro grafové databáze zatím neexistuje jeden standardní jazyk. Lze narazit na jazyky vyvinuté specificky pro konkrétní systém. Příkladem může být jazyk *Cypher* vyvíjený společností Neo4j nebo *AQL* využívaný v databázi ArangoDB. Na druhou stranu jsou zde jazyky, jakým je například *Gremlin*, které lze použít ve více systémech zacházejících s grafy. Mezi další známé jazyky patří:

- *SPARQL* – využíváný pro znalostní grafy v RDF formátu
- *PGQL* – je jednou z komponent datové databáze společnosti Oracle, který je postavený na SQL
- *GSQL* – procedurální jazyk používaný v databázi TigerGraph podobný SQL
- *GraphQL* – jedná se o dotazovací jazyk, který není závislý na konkrétní implementaci. Za jeho vývojem stojí Facebook. Je považován za přímého konkurenta REST API.

2.7.1 Cypher

[12], [13] Cypher je inspirovaný známým jazykem SQL z relačních databází. Důkazem může být použití několika stejnojmenných klíčových slov, jako je WHERE, LIMIT, ORDER BY apod. Navíc v obou případech se jedná o deklarativní dotazovací jazyky. Myšlenkou deklarativních jazyků je definování, jaká data chceme z databáze dostat, místo definování jak je dostat. Cypher je unikátní v tom, že využívá kulaté závorky pro vrcholy a hranaté pro hrany, čímž docílí lepší přehlednosti v dotazu. Základní syntaxe pro průchod grafem z vrcholu *vrchol1* na vrchol *vrchol2*, přes hranu *HRANA*, která je spojuje, je ve formátu

$$(vrchol1)-[:HRANA]->(vrchol2).$$

Obdobně jako v SQL je základní dotaz v Cypheru složen ze tří částí. První je povinná a začíná klíčovým slovem MATCH. Určuje, které vrcholy a hrany se

⁷<https://json-schema.org/>

mají vybrat. Část MATCH je často následována částí začínající klíčovým slovem WHERE. Ta je nepovinná. Filtruje vybrané elementy z první části dotazu na základě jejich vlastností. Poslední, povinná, část začíná klíčovým slovem RETURN. Definuje, co je po provedení dotazu vráceno. Na výsledky lze následně aplikovat další operace, jakými jsou například agregování, limitování, nebo seřazování.

Pro úsporu paměti implementuje vždy, kde je to možné, *líné vyhodnocování dotazů* (anglicky lazy evaluation). Tedy u většiny operátorů nečekáme na všechny výsledky aplikace daného operátoru, ale hned po jejich získání na ně aplikujeme další nadřazený operátor. Výpočet podřazeného operátoru tedy obecně nemusí skončit dříve, než na výsledky začneme aplikovat nadřazený operátor. Tím nedochází k zatěžování paměti mezivýsledky výpočtu. Nicméně ne všechny dotazy lze vyhodnocovat líně. Pro operace jako je třídění, nebo agregování je nutné použít *striktní vyhodnocování dotazů* (anglicky eager evaluation). S výsledky pak nadřazený operátor manipuluje až po úplném skončení výpočtu.

2.7.2 Gremlin

[16], [17]

Gremlin je dotazovací jazyk pro procházení grafů, který je součástí grafového výpočetního frameworku Apache TinkerPop⁸. Gremlin lze použít pro všechna grafová uložistě, která tento framework podporují. Jedná se o imperativní jazyk s nějakými deklarativními prvky. Podobně jako Cypher ve většině případů vyhodnocuje dotazy líným způsobem.

Apache TinkerPop mimo jiné obsahuje také Gremlin Console, což je textové REPL (Read-Eval-Print Loop) rozhraní pro práci s lokálními, či vzdálenými grafy. Je založená na Groovy konzoli. Groovy je programovací jazyk, který je označován jako nadmnožina Javy. Co funguje v Javě funguje i v Groovy. V konzoli tedy můžeme používat validní kód napsaný v tomto jazyce. Díky tomu lze například uložit do proměnné určitý vrchol a na něj se pak přes danou proměnnou odkazovat. Konzole je široce rozšiřitelná pomocí různých pluginů.

Pro napojení na databázi OrientDB budeme potřebovat plugin *tinkerpop.orientdb*. Když je tento plugin aktivovaný, lze v konzoli vytvořit novou instanci *OrientGraph*. Uložíme ji do proměnné graph. Při vytváření je nutné specifikovat URL pro připojení k databázi. Připojit se můžeme k lokální databázi, vzdáleně k databázovému serveru, nebo k jednorázové databázi v paměti. Může být též potřeba zadat přihlašovací údaje.

Než začneme psát dotazy pomocí Gremlinu musíme ještě vytvořit instanci zdrojového objektu pro procházení grafu zavoláním metody *traversal()* na hodnotu v proměnné graph. Celý postup je uveden ve zdrojovém kódu 1.

2.7.3 AQL

[15] Jedná se převážně o deklarativní dotazovací jazyk používaný v databázi ArangoDB. Snaží se být nezávislý na klientovi (client-independent). To znamená,

⁸<http://tinkerpop.apache.org/>

```

1 //OrientDb database connector
2 graph = OrientGraph.open(url, user, password);
3
4 //establishing a graph traversal source object
5 g = graph.traversal();

```

Zdrojový kód 1: Připojení se k databázi OrientDB v Gremlin Console.

že jazyk a syntaxe není závislá na programovacím jazyku, který klient využívá. Obdobně jako velká část grafových dotazovacích jazyků je ovlivněný jazykem SQL. Jazyk je jednotný pro všechny databázové modely.

AQL je čistě DML (Data Manipulation Language) jazyk. Tedy umí číst a manipulovat s daty v kolekcích. Neumí vytvářet a odstraňovat databáze, kolekce a indexy.

Každý dotaz v AQL musí buď vracet výsledky pomocí klíčového slova *RETURN*, nebo provádět operaci, která modifikuje data. Je podporován pouze jeden dotaz v rámci jednoho dotazovacího řetězce. Není možné provést výpočet více oddělených dotazů pro jedno zavolání. Dotaz je opět vyhodnocován líně, pokud se v něm neobjevují poddotazy, ty jsou ve výrazech vyhodnocovány předem, příkladem je ternární operátor.

Dotazy lze klást prostřednictvím webového rozhraní, nebo konzole arangosh. V práci se dotazy kladly pouze prostřednictvím první zmíněné možnosti.

3 Modelování grafových dat

Tato kapitola byla napsána s využitím knih [18], [5], [19]. Dále byly využity tyto zdroje [11], [20] a [13].

POZNÁMKA 17 (PŘEKLAD)

V práci je použito české slovo „vlastnosti“ přeložené z anglického slova „properties“, se kterým se běžně uživatel setká při práci s grafovou databází. Slovo „label“ je z důvodu znělosti zachováno v původním znění.

Modelování dat je proces, který překládá entity reálného světa a vztahy mezi nimi do jejich softwarové reprezentace. Uživatel popisuje doménu jako graf skládající se z vrcholů a hran pomocí vlastností a labelů. Každý model je unikátní. Podoba výsledného modelu vždy závisí na případech použití a otázkách, které mají podobu dotazů, které budou na databázi kladeny. Dodržení základních principů modelování dat je důležité pro výslednou efektivitu. Když je graf přizpůsoben dotazům, dokáže velmi rychle nalézt potřebnou odpověď. Naopak špatně zvolený model grafu může ve výsledku aplikaci ublížit a být překážkou. Například tím, že zpracování dotazu bude trvat déle než u optimalizovaného modelu. Další důležitou věcí, na kterou je potřeba myslet, je budoucnost vyvíjené apli-

kace. Model by měl být připravený na změny, které po aplikaci budou její koncoví uživatelé požadovat. Správnému modelu je proto dobré věnovat dostatečné množství času a dodržovat obecná pravidla při jeho vytváření.

Je potřeba také podotknout, že ne všechna data jsou vhodná pro uchovávání v grafově podobě. Rozhodnutí, zda pro naše účely je grafová databáze dobrým řešením, je ve spoustě případů náročné. Existují různé typy databází (například relační, dokumentové, nebo klíč-hodnota), kde každá má své silné a slabé stránky a je potřeba si dobře promyslet, kterou zvolit. Lze dohledat spoustu „slovních návodů“ pomáhajících s touto volbou. Na obrázku 3 je ukázán rozhodovací strom převzatý z [18]. Grafově databáze jsou výhodné pro aplikace, které mají vztahy mezi entitami na prvním místě. Tato vlastnost by měla být odrazovým můstkem při rozhodování, zda použít, či nepoužít grafovou databázi.

Pokud se pro požití grafově databáze rozhodneme, můžeme přistoupit k modelování dat. V této kapitole se tedy zaměříme na návrh grafového modelu. Zkusíme si to na příkladu sociální platformy pro nadšence cestování. Sociální sítě jsou totiž jedním z případů, kdy použití grafových databází pro ukládání dat dává smysl. Tato aplikace tedy bude umožňovat cestovatelům se propojit s ostatními. Dále bude zaznamenávat jejich výlety do různých destinací, hodnocení těchto cest, cestovatelské aktivity, které destinace nabízí a které se mohou uživatelům aplikace líbit. Na základě svých přátel a hodnocení cest do destinací aplikace dokáže uživateli doporučit jiné destinace, které dosud nenavštívil. Naše aplikace se tedy bude dotýkat i dalšího případu užití grafových databází, a to systémů pro doporučování, které jsou v praxi hojně využívány. Pro vyzkoušení dotazů můžeme využít csv souborů, obsahujících vygenerovaná data pro naši aplikaci. Seznam všech souborů, rozdělený na entity a vztahy mezi nimi a počet řádků (včetně hlavičky) jednotlivých souborů lze vyčíst z tabulky 1.

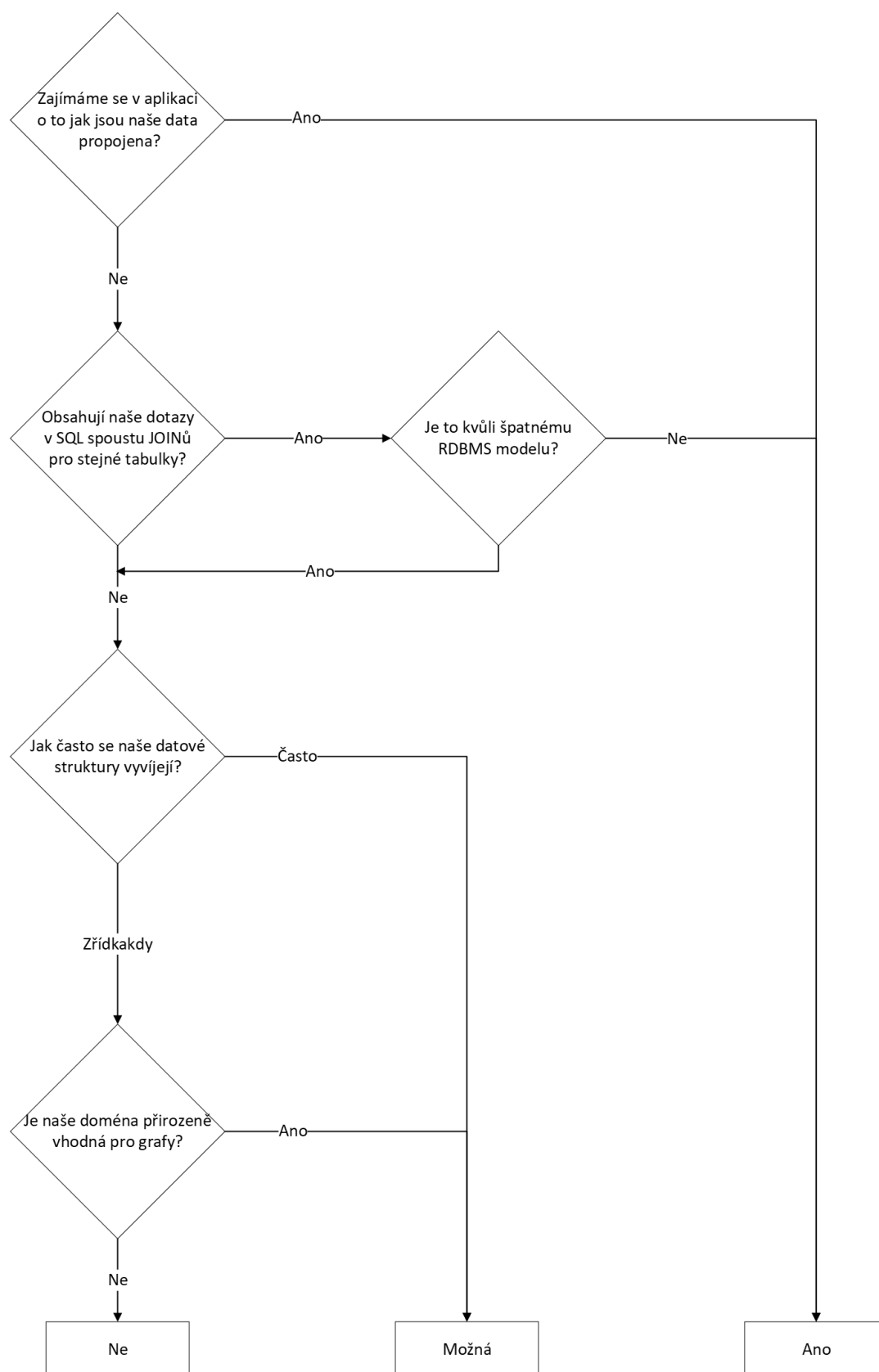
Postup modelování se dá rozdělit do čtyř fází:

1. Definice problému
2. Konceptuální datový model
3. Logický datový model
4. Testování modelu

V následujících částech se na jednotlivé fáze podíváme detailněji.

3.1 Definice problému

V první fázi bychom měli zvážit co všechno by měla aplikace umět. Výsledkem by měl být její, co nejpřesnější, slovní popis. Pomoci nám může odpovědět na těchto pár otázek.



Obrázek 3: Rozhodovací strom pro vhodnost použití grafové databáze.

Tabulka 1: Tabulka souborů pro aplikaci pro cestovní nadšence.

Typ	Název souboru	Počet řádků	Informace o souboru
Entity	activities.csv	49	Aktivity, které mohou destinace nabízet.
	countries.csv	58	Státy, v kterých se vyskytují jednotlivé destinace.
	destinations.csv	204	Destinace, které mohou uživatelé navštívit.
	journeys.csv	297	Cesty, v rámci kterých uživatelé navštěvují destinace.
	ratings.csv	641	Hodnocení jednotlivých cest uživateli.
	users.csv	301	Uživatelé sociální sítě Travel Enthusiast Network.
Vztahy	follows.csv	5738	Vztah mezi uživateli.
	is_within.csv	204	Vztah mezi destinacemi a státy.
	led_to.csv	1442	Vztah mezi cestami a destinacemi.
	likes.csv	1517	Vztah mezi uživateli a cestovními aktivitami.
	lives_in.csv	301	Vztah mezi uživateli a destinacemi.
	offers.csv	2023	Vztah mezi destinacemi a cestovními aktivitami.
	participated.csv	1518	Vztah mezi uživateli a jejich cestami.
	wrote_about.csv	641	Vztahy mezi uživateli, recenzemi a cestami.

3.1.1 Co od aplikace očekáváme?

Aplikace je sociální síť, a proto by měla jednotlivým uživatelům umožňovat se propojit s ostatními uživateli aplikace. To je umožněno pomocí relace sledování. Uživatelům by pak na základě jimi sledovaných uživatelů, oblíbených aktivit a hodnocení destinací měly přicházet tipy na výlet. Aplikace by měla umožnit záznamy, že někteří uživatelé podnikli cestu společně. Uživatelé budou moci v rámci jedné cesty navštívit více destinací.

3.1.2 S jakými informacemi aplikace pracuje?

- Všechny základní informace o *uživateli*, aby se s nimi ostatní mohli spojit - jejich jméno, příjmení, unikátní identifikátor, oblíbené *cestovatelské*

aktivity.

- Informace o *cestách*, které uživatelé podnikli.
- Informace o konkrétních *destinacích* jako je jejich lokace, název, cestovatelské aktivity, které destinace nabízí.
- Uživateli je umožněno psát *hodnocení* jednotlivých *cest*, které podniknul.

V reálném scénáři by projekt pravděpodobně zahrnoval mnoho dalších informací, ale pro tuto ukázkovou aplikaci to ponecháme omezené. Pro zjednodušení se vyhneme přesným souřadnicovým informacím a vystačíme si s informací v rámci jakého státu se daná destinace nachází. Dále uživatelům aplikace bude dovolovat psát hodnocení na dané cesty, nikoliv na destinace, které v rámci cesty podnikli. Uživatelé cestující spolu se budou účastnit stejné cesty do destinací.

Aplikace by tedy měla zacházet s pojmy jako jsou uživatel, destinace, stát, cesta, hodnocení a cestovní aktivita.

3.1.3 Jak spolu tyto informace vzájemně souvisí?

Následující seznam ukazuje, jakým způsobem jsou odpovědi na předchozí otázku mezi sebou propojeny.

- Uživatel *sleduje* jiného uživatele.
- Uživatel *se zúčastnil* cesty *vedoucí do* destinace.
- Uživateli se *líbí* cestovatelská aktivita.
- Uživatel *napsal* recenzi *o* cestě.
- Destinace *nabízí* cestovatelskou aktivitu.
- Destinace *je ve* státě.

3.1.4 Na jaké otázky potřebujeme v aplikaci najít odpověď?

- Jaké všechny uživatele sleduji? A koho sledují oni? Které uživatele nesleduji, ale sledují je uživatelé, které sleduji?
- Najdi 3 nejlépe hodnocené destinace, které jsem nenavštívil, nabízející konkrétní cestovní aktivity?
- Které destinace navštívil uživatel X? Kolikrát je navštívil?
- Najdi mi všechny uživatele v rámci skupiny, s kterou cestuje uživatel X do destinace Y.
- Najdi destinace nabízející některé z cestovatelských aktivit, které se uživateli líbí. Seřaď je na základě hodnocení.

- Pro konkrétní destinaci vrať měsíc v roce v kterém je nejlépe hodnocená.

V reálném scénáři by otázek mohlo být mnohem více.

3.2 Konceptuální datový model

Dalším krokem je vyvinout konceptuální model. Výstupem by měl být přehledný diagram popisující typy dat v naší aplikaci. Vyvarujeme se v něm pro nevývojáře zbytečným detailům a zaměříme se pouze na to, jak data různých typů v rámci vyvíjené aplikace spolu souvisí. Tento proces vyžaduje identifikování, seskupení a propojení všech entit a vztahů, které se v naší aplikaci vyskytují. Nejedná se o jednoduchý úkol. Je možné, že jej navrhujeme špatně a budeme muset jej předělat.

Při vyvíjení datového modelu vývojáři často kreslí na tabuli, jak jsou data v rámci projektu propojena s dalšími. Proto se konceptuální model v různých literaturách nazývá whiteboard model.

3.2.1 Entity

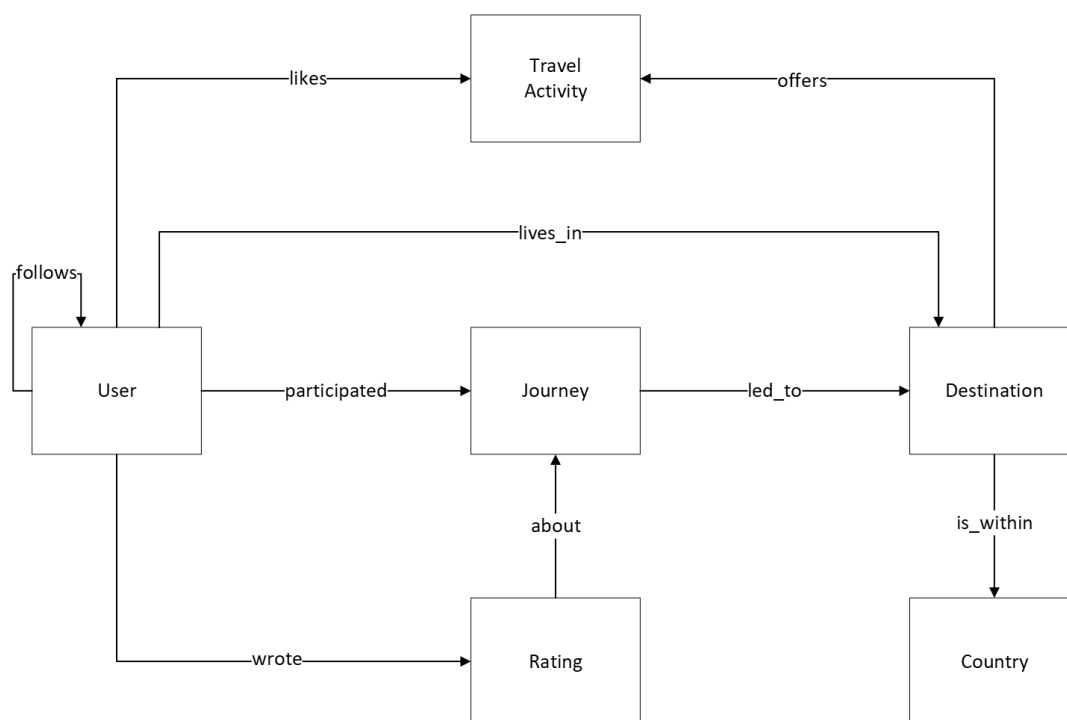
Entity většinou představují objekty, jakými jsou například *Uživatel*, *Účet*, nebo *Město*, které jsou v databázi uloženy a propojeny vzájemnými vztahy.

Jedním z osvědčených postupů, jak z našich poznámek z minulé fáze vytáhnout entity, je dívat se po podstatných jménech. Pokud se budeme řídit tohoto postupu, tak z věty „Uživateli se líbí cestovatelská aktivita.“ získáme dvě entity, a to *Uživatel* a *Cestovatelská aktivita*. Avšak ne všechna podstatná jména jsou nutně entity. Příkladem může být podstatné jméno „příjmení“. Příjmení v naší aplikaci chápeme jako součást uživatele. A v této fázi se jedná o nepodstatný detail.

V naší aplikaci můžeme dohromady rozpoznat 6 různých entit: *Uživatel*, *Cesta*, *Destinace*, *Aktivita*, *Stát* a *Recenze*.

3.2.2 Vztahy

Nyní následuje propojení entit pomocí vztahů. Entity v rámci naší aplikace jsme identifikovali zejména zaměřením se na podstatná jména z poznámek o aplikaci. Vztahy můžeme získat obdobně pomocí sloves. Zde bychom však měli být velmi opatrní. Ve slovese se často ukrývá podstatné jméno. V budoucím vývoji aplikace by pak mohlo dojít k problémům při rozšiřování funkcionality aplikace. Dobrým příkladem je sloveso „recenzovat“. Pokud bychom při modelování zvolili větu „Uživatel recenzoval X.“ místo „Uživatel napsal recenzi o X.“, tak bychom mohli v budoucím vývoji aplikace narazit na problém, že nemáme entitu představující recenzi, na kterou bychom mohli odkazovat. Snadno si jde domyslet komentování nebo jiný způsob reagování na recenze. Podobně, pokud bychom v naší aplikaci upřednostnili větu „Uživatel navštívil destinaci.“ před větou „Uživatel podnikl cestu do destinace.“, uživatel by pak nemohl danou cestu hodnotit tak, jak to



Obrázek 4: Konceptuální datový model pro sociální síť pro cestovní nadšence.

máme v popisu naší aplikace. Takové chyby mohou vést v budoucnosti k migrování databáze, což je velmi obtížný a zdoluhavý proces přesouvání dat z jedné databáze do druhé, kterému je lepší se vyhnout.

Hledáme tedy slovesa, které představují relaci mezi námi nalezenými entitami. Po aplikaci tohoto doporučení na větu „Uživatel se líbí cestovatelská aktivita.“ získáme vztah *se líbí* mezi entitami *Uživatel* a *Cestovatelská aktivita*. Stejným způsobem postupujeme dál a hledáme všechny důležité vztahy. Může se stát, že při důkladném hledání vztahů nalezneme další entity, které jsme předtím nenalezli. Můžeme se dostat do situace, kdy váháme, zda určitý typ dat má tvořit entitu nebo relaci v našem modelu. V této situaci je obecně doporučeno se rozhodovat podle toho, zda některý dotaz na databázi dostane na vstupu informaci ve formě tohoto typu dat. Pokud ano, tato data bychom měli reprezentovat jako entitu. Například lze takto odvodit, že slovo Uživatel musí být entita, protože je vstupem (proměnnou) v dotazu odpovídajícím na otázku „Které destinace navštívil uživatel X? Kolikrát je navštívil?“.

3.2.3 Konceptuální model

Po důkladném rozmyšlení má naše tabule stejný obsah, jako na obrázku 4.

3.3 Logický datový model

Pokud jsme s diagramem z předchozí fáze spokojeni, můžeme konečně přistoupit ke kroku, ve kterém definujeme základní schéma, které bude určovat podobu našich dat v databázi. V případě relačního modelování je konceptuální model přeformátován, aby mohl být ukládán do tabulek. V případě grafového modelování toto není potřeba. Stačí identifikovat vrcholy a hrany a přiřadit jim příslušný label a vlastnosti. Výstupem tedy bude opět diagram, který ale bude mnohem detailnější. Bude se zabývat detaily, kterými se zabývají vývojáři. Bude definovat tvar, ve kterém budou data v databázi uložena a podle něj budeme moci klást na databázi dotazy. Poté již můžeme do naší databáze vložit zkušební data a vyzkoušet dotazy, které budou odpovídat na naše otázky načrtnuté v první části modelování dat.

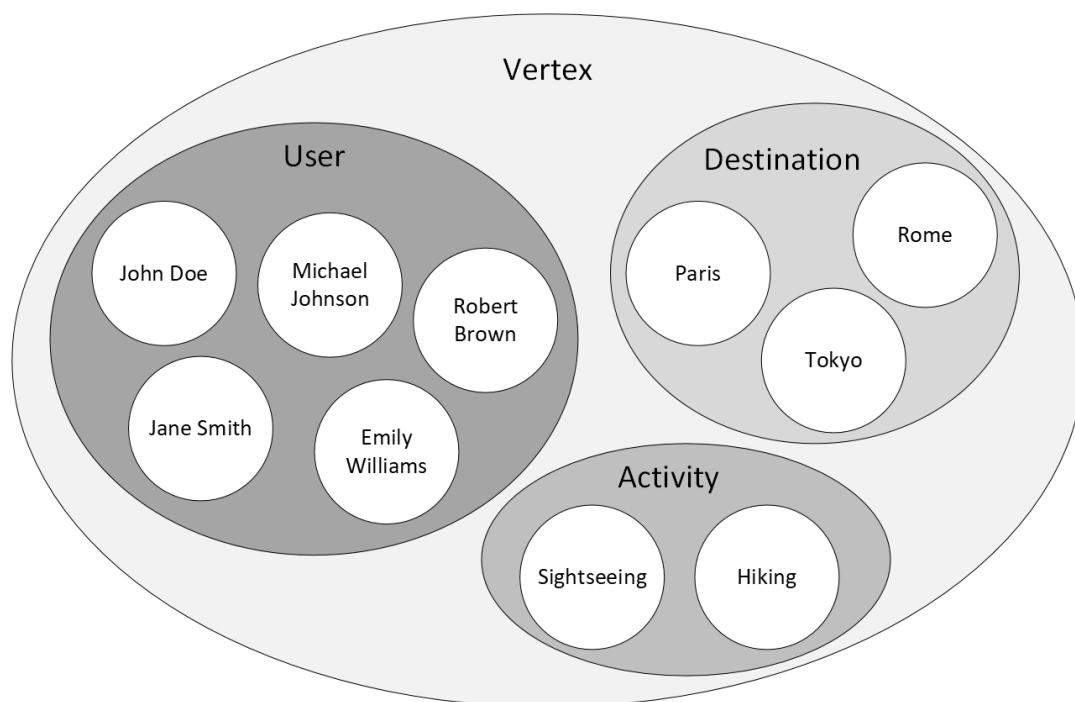
Pro vytvoření grafového modelu jsou potřeba tyto 3 kroky:

1. Převést entity na vrcholové labely
2. Převést relace na labely hran
3. Najít a přiřadit vlastnosti vrcholovým a hranovým labelům

3.3.1 Vrcholové labely

Podstatnou část práce již máme za sebou, neboť můžeme využít dobře navrženého konceptuálního modelu a namapovat naše entity na vrcholové labely. Label vrcholu u určuje, do které skupiny, nebo kategorie vrcholů vrchol u patří. Všechny vrcholy se stejným labelem pak spadají do stejné kategorie. Label pak můžeme používat v dotazech na databázi a pracovat tak s různými skupinami vrcholů a dosáhnout tak lepšího výkonu při dotazování. Pro lepší pochopení je zde obrázek 5. Vymyšlení vrcholových labelů, tedy názvů jednotlivých skupin vrcholů, může být poněkud náročný úkol. Labely by měly být výstižné, krátké a přesné. Musíme mít na paměti budoucí rozšiřitelnost a fakt, že s databází může pracovat jiný vývojář. Dobře zvolený label je tedy velmi žádaný.

Dobrým zvykem je zvolit label, který bude v jednotném čísle. Při přiřazování labelu je dobré myslet na jeho co největší obecnost, protože v budoucnu bychom mohli potřebovat reprezentovat jiný podobný koncept a tedy je užitečné volit co nejobecnější label. Povolený počet labelů pro jeden vrchol se u různých grafových databází liší. Bezpečným přístupem je každému vrcholu přiřadit jeden label. To je například přístup Apache TinkerPop. Často se ale také hodí mít labelů více a pomocí nich umět vyjádřit dědičnost. Vrcholy pak spadají do více skupin vrcholů naráz. Příkladem může být systém pro správu zaměstnanců a klientů v nějaké firmě. Zde by se mohlo hodit kromě labelu *Zaměstnanec* a *Klient* mít label *Osoba*. Neboť zaměstnanec, reprezentovaný vrcholem v databázi, jistě sdílí s klientem informace jako je jméno, datum narození apodobně, ale může mít nějaké dodatečné informace, jako je například plat, nebo identifikační číslo zaměstnance. Nyní jednoduše můžeme vybrat data týkající se pouze



Obrázek 5: Label vrcholu definuje, do které kategorie vrchol spadá.

klientů, ale zároveň můžeme požádat databázi o informaci o všech lidech spravovaných v databázi. Příklad přiřazení více labelů jednomu vrcholu je znázorněn na obrázku 6. To je důvod, proč databáze, jako Neo4j nebo Amazon Neptune umožňují vrcholům přiřadit žádný, jeden, ale i více labelů.

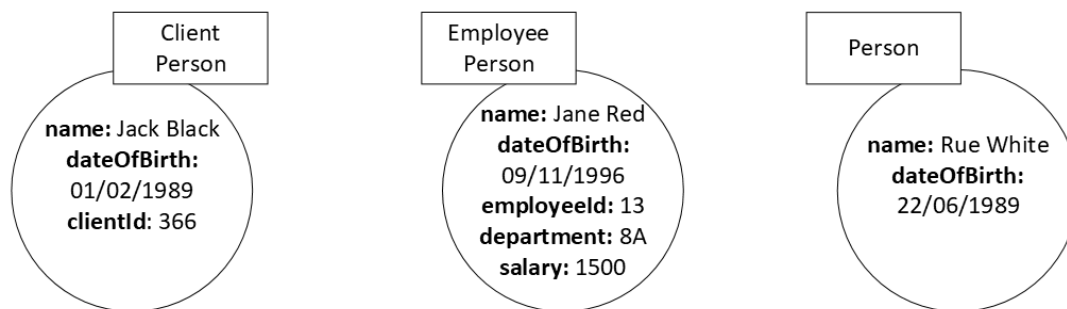
Častou chybou při navrhování labelů je nekonzistentní pojmenovávací konvence. Proto je dobré si nyní stanovit pojmenovací konvenci, kterou se budeme řídit.

POZNÁMKA 18 (KONVENCE POJMENOVÁNÍ)

V našem příkladu je zvolena konvence doporučená v dokumentaci databáze Neo4j a tedy pro labely vrcholů budeme používat *UpperCamelCase* notaci, pro labely hran *SCREAMING_SNAKE_CASE* notaci s použitím podtržítka pro oddělení slov a pro vlastnosti camelCase notaci.

3.3.2 Hranové labely

Při hledání hran opět využijeme připraveného konceptuálního modelu. Podobně jako u vrcholů se zamyslíme nad labelem hran, které by opět mělo být obecné a výstižné. Navíc bychom se měli držet jedné dobré zásady. Přečtením labelu počátečního vrcholu, labelu hrany z něj vycházející a labelu cílového vrcholu do nějž hrana vstupuje by nám mělo dávat vcelku smysluplnou větu. V češtině však musíme opomenout skloňování. Příkladem by mohla být věta „Uživatel napsal



Obrázek 6: Ukázka více labelů pro jeden vrchol.

hodnocení.“, která vznikla spojením vrcholového labelu *Uživatel*, hranového labelu *napsal* a labelu cílového vrcholu *Hodnocení*. Ve většině případů mají věty tvar podle následující posloupnosti větných členů – podmět-přísudek-předmět.

To nás přivádí k orientaci hran. Předchozí rada nám rovnou specifikuje i orientaci hrany. Tedy z věty „Uživatel napsal hodnocení.“ máme, že hrana s labelem *napsal* vede z vrcholu s labelem *Uživatel* do vrcholu s labelem *Hodnocení*, nikdy ne naopak. Obrácením hrany bychom přečtením vrchol-hrana-vrchol dostali nesmyslnou větu „Hodnocení napsalo uživatele.“. Pokud bychom v tomto případě požadovali opačnou orientaci hran, museli bychom přistoupit ke změně labelu hrany. Tedy například hraně s labelem *napsal* bychom změnili label na *bylo napsáno* a pak by věta „Hodnocení bylo napsáno uživatelem.“ dávala smysl. Orientace hrany by měla reflektovat to, jak přirozeným způsobem přemýšlíme o datech v naší doméně.

Nyní zbývá ještě určit *jedinečnost* hran. Jedinečnost lze definovat jako počet hran se stejným labelem mezi dvěma vrcholy. Tyto vrcholy mohou a nemusí mít stejný label.

Jedinečnost hran se dělí na dva typy:

- *jednoduchá*
- *a mnohonásobná.*

Jednoduchá jedinečnost odkazuje na žádnou, nebo právě jednu hranu zatímco vícenásobná povoluje hran se stejným labelem mezi dvěma vrcholy více. Na jednoduchou jedinečnost hrany s konkrétním hranovým labelem lze též nahlížet tak, že skupina sousedních vrcholů, do kterých se dostaneme přes hrany s tímto labelem, neobsahuje více instancí jednoho vrcholu, tedy tvoří množinu vrcholů. Zatímco u mnohonásobné jedinečnosti dostaneme kolekci sousedních vrcholů, kde se vrcholy mohou opakovat. Co se týče jedinečnosti hran v naší aplikaci tak všechny hrany, až na hrany mezi vrcholy s labelem *Cesta* a *Destinace* budou mít jedinečnost jednoduchou. Například vztah mezi uživatelem a cestovní aktivitou s labelem *LIKES* buď je nebo není a to lze vyjádřit pomocí existence jedné hrany mezi těmito dvěma vrcholy. Totéž platí pro ostatní hranové labely, až na hranu s

labelem *LED_TO*. Jistě totiž může existovat situace, kdy v rámci cesty uživatel do dané destinace zavítá vícekrát a s touto možností by náš model měl také počítat. Tedy tato hrana bude mít vícenásobnou jedinečnost. Tyto hrany pak od sebe rozlišíme pomocí vlastností, které budou určovat období, kdy se daný uživatel v dané destinaci nacházel. Hrany s mnohonásobnou jedinečností bychom měli rozlišovat pomocí vlastností, aby byly unikátní, jinak tato jedinečnost postrádá smysl.

Zvolení správné jedinečnosti hran je klíčové pro správné chování databáze. Pokud zvolíme jednoduchou jedinečnost tam, kde by měla být vícenásobná, databáze může vracet méně dat než očekáváme. Naopak, vícenásobná jedinečnost hran tam, kde by měla být jednoduchá, může vést k duplikativním datům a sníženému výkonu dotazů. Každá hrana se stejným labelem mezi dvěma vrcholy zvyšuje čas výpočtu a komplikuje průchod grafem.

U některých implementací grafových databází, jako například DataStax Graph nebo JanusGraph, lze tuto jedinečnost vynutit jako součást při definování schématu. U implementací bez schématického přístupu, jako je Neo4j, tuto jedinečnost lze ošetřit pouze v rámci aplikace.

3.3.3 Vlastnosti

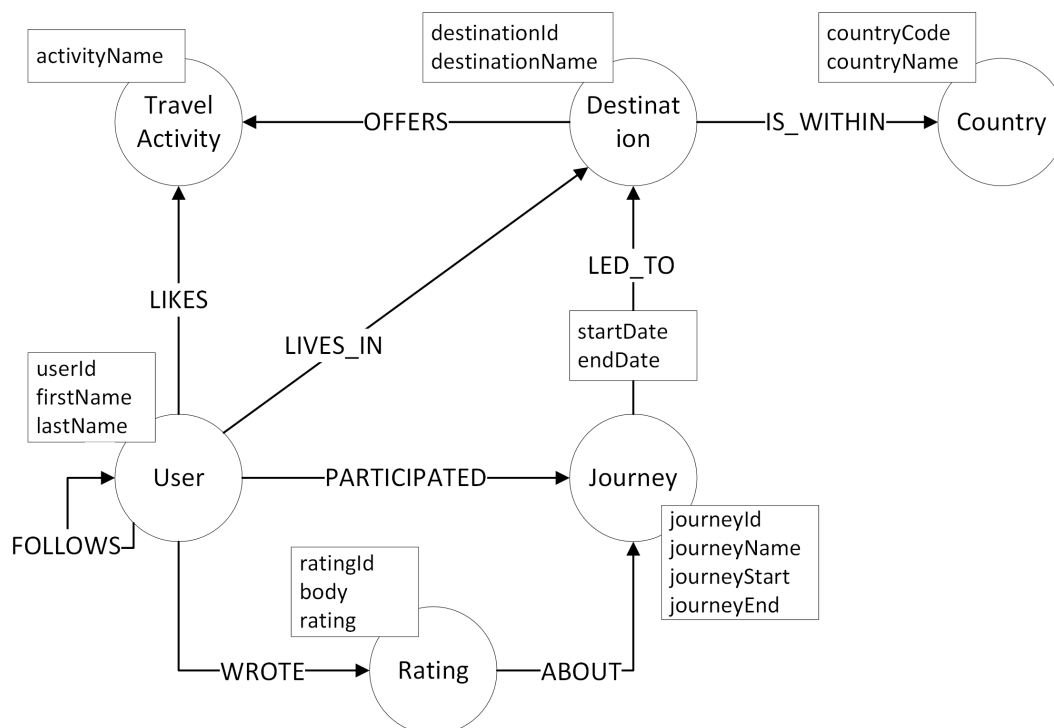
Poslední zastávkou při konstruování logického modelu jsou vlastnosti. Základní kameny grafu vrcholy a hrany máme, ale chybí nám způsob jak rozlišit jednotlivé vrcholy se stejným labelem. K tomu nám slouží vlastnosti. V grafových databázích mohou mít vlastnosti jak hrany tak vrcholy, neboť hrany jsou pro nás stejně významné jak vrcholy.

Vlastnosti v grafové databázi jsou páry klíč-hodnota a popisují konkrétní atributy hrany, či vrcholu. Hodnota vlastnosti může být různého datového typu, například řetězec, číslo, pravdivostní hodnota nebo datový typ pro uchování datumu a času. Někdy též může být hodnota typu pole nebo seznam.

Pomocí vlastností následně můžeme rozlišovat jednotlivé uživatele například pomocí jejich křestního jména, příjmení, nebo uživatelské přezdívky. To se nám hodí při filtrování dat, když chceme vybrat pouze vrcholy s určitou vlastností. Jednou z vlastností bývá unikátní identifikátor každého vrcholu, díky kterým přistupujeme ke konkrétním vrcholům.

K dokončení logického modelu nám tedy zbývá najít a správně přiřadit potřebné vlastnosti konkrétním vrcholovým a hranovým labelům, rozumně je pojmenovat a určit jejich datový typ. K jejich nalezení nám opět mohou pomoci otázky, které chceme aby nám databáze zodpověděla. Z nich můžeme jednotlivé vlastnosti identifikovat a přiřadit.

Naše výsledné grafové schéma popisuje obrázek 7. Vrcholové labely jsou uvedeny v kroužcích. Hranové labely jsou vepsány do šipek. Vlastnosti vrcholů a hran jsou zobrazeny v obdélnících nacházejících se u vrcholů a na hranách.



Obrázek 7: Logický datový model pro sociální síť pro cestovní nadšence.

3.4 Testování modelu

Testování našeho grafového modelu spočívá ve vyzkoušení, zda se chová dle našeho očekávání (databáze vrací očekávané výsledky) a dotazování není moc pomalé ani pro velké množství dat. K testování lze využít široké škály nástrojů, které dotazovací jazyky a databáze nabízejí.

3.4.1 Unit testy

Při testování správnosti dotazů je vhodné použít tzv. *unit testy*. Tedy vytvoříme si pro každý dotaz malé množství testovacích dat (například kolem 10 vrcholů, tak abychom z něj pouhým okem vyčetli správnou odpověď), kterými otestujeme správnost odpovědí z databáze. Příkladem unit testu je zdrojový kód 2. Tento kód vloží do databáze data, která tvoří graf znázorněný na obrázku 8. Následně zkontroluje dotaz hledající všechny uživatele, které mají od uživatele s vlastností `userId` rovnou 1 vzdálenost rovnou 2. Vidíme, že po spuštění testu databáze by měla vrátit uživatele s identifikátory 4 a 5. Následně po sobě uklidíme. Smažeme testovací data.

```

1 // Vytvoření testovacích dat
2 CREATE (u1:User {userId: 1, firstName: "John", lastName: "Doe"})
3 CREATE (u2:User {userId: 2, firstName: "Lisa", lastName: "Fog"})
4 CREATE (u3:User {userId: 3, firstName: "Bob", lastName: "John"})
5 CREATE (u4:User {userId: 4, firstName: "Jane", lastName: "Doe"})
6 CREATE (u5:User {userId: 5, firstName: "Paul", lastName: "Bae"})
7
8 CREATE (u1)-[:FOLLOWS]->(u2)
9 CREATE (u2)-[:FOLLOWS]->(u3)
10 CREATE (u1)-[:FOLLOWS]->(u3)
11 CREATE (u2)-[:FOLLOWS]->(u4)
12 CREATE (u3)-[:FOLLOWS]->(u5)
13
14 // Spuštění testu
15 MATCH (u:User {userId: 1})-[:FOLLOWS]->(u2)-[:FOLLOWS]->(u3)
16 WHERE NOT EXISTS((u)-[:FOLLOWS]->(u3)) AND u3 <> u
17 RETURN u3;
18
19 // Smazání testovacích dat
20 MATCH (u:User) WHERE u.userId IN [1, 2, 3, 4, 5]
21 DETACH DELETE u;

```

Zdrojový kód 2: Unit test.

3.4.2 Profile a Explain

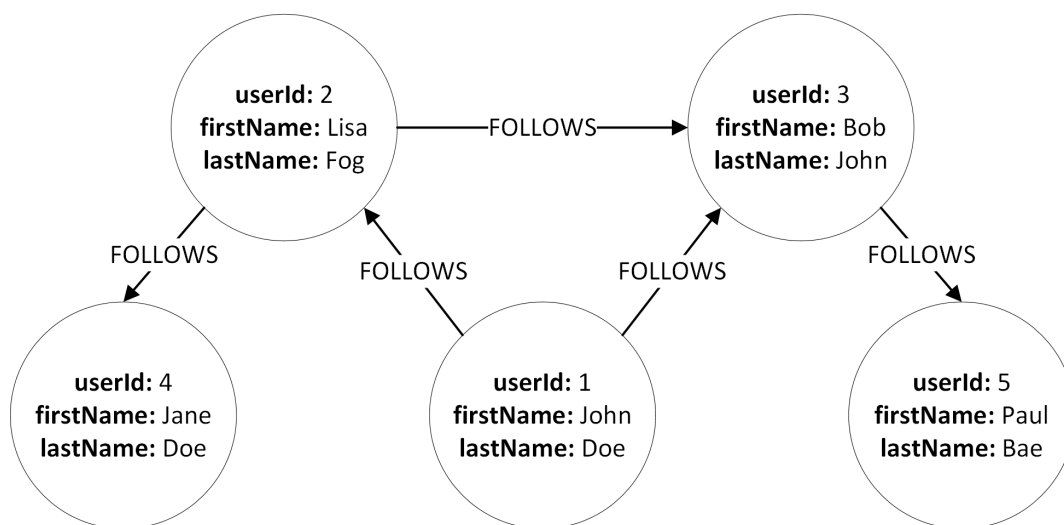
Pokud čelíme pomalému vykonávání některých dotazů, můžeme použít dva nástroje pro diagnostiku problémů.

První z nich se nazývá *explain* a slouží pro vysvětlení jak se bude průchod grafem vyvíjet na základě našeho dotazu. Používá se v situacích, kdy chceme zjistit chování průchodu grafem, ale nechceme jej vykonat. Výsledkem je optimalizovaný exekuční plán daného dotazu, který by byl vykonán na datech.

Druhému říkáme *profile* a podle dodaného dotazu profiluje, co průchod grafem udělal. Profile je hojně používaným nástrojem pro analýzu dotazů, neboť dotazy jsou často pomalé pouze pro několik vrcholů – například těch, které mají vysoký stupeň. Toho lze využít k porovnání chování rychlých a pomalých vykonání dotazu. Získané informace pak využijeme k optimalizování našich dotazů.

3.4.3 Indexy

Pro efektivní hledání dat v rámci grafu lze použít *indexy*. Toho lze využít pro vylepšení dotazů, neboť díky nim můžeme zrychlit průchod grafem, přístup ke konkrétním datům a další operace pracující s daty. Indexy vytvoří redundantní kopii vybraných dat v databázi podle zadaných kritérií. Jsou výhodné pro data, které jsou častým terčem hledání. Nejčastěji se jedná o vlastnosti, které často vystupují ve filtrování dat v dotazu, nebo identifikátory vrcholů. Dále se využívají pro vlastnosti, na kterých jsou prováděny různé operace s řetězci. Příkladem je



Obrázek 8: Graf vytvořený pomocí zdrojového kódu [2](#).

hledání slov začínající, končící, nebo obsahující určitý podřetězec.

Implementace jako je TinkerGraph poskytuje pouze globální indexy jedné hodnoty. Oproti tomu databáze jako Neo4j nabízí celou škálu různých typů indexů. Indexy také však mají nevýhody. Zvyšují nároky na úložný prostor a mohou zpomalovat zápisy do databáze. Měly by tedy být vytvářeny pouze pro data, pro které budou obzvláště výhodné.

3.4.4 Denormalizace

Denormalizace je metoda, která do databáze přidává redundantní data za účelem zrychlení čtení dat. Denormalizace v grafových databázích většinou znamená kopírování hodnot vlastností vrcholů na incidentní hrany. Často se jedná o časové informace. Nevýhodou tohoto procesu je udržba těchto kopií což vede k zpomalení zápisů. Denormalizace je tedy vhodná pouze pro aplikace, kde počet operací pro čtení z databáze mnohonásobně převyšuje počet operací pro zápis.

4 Known walk

Často při průchodu grafem předem známe grafový vzor určující cestu, kterou musíme projít, abychom dosáhli požadovaných výsledků pro daný dotaz. Předem víme délku cesty a vrcholové a hranové labely, přes které cesta prochází. Tento průchod grafem je v knize [\[18\]](#) označen jako *known walk*. Při překladu otázek na dotazy v naší aplikaci pro cestovatelské nadšence s datovým modelem na obrázku [7](#) zjistíme, že všechny grafové vzory představují *known walk*.

4.1 Příklad použití

Příkladem může být dotaz odpovídající na otázku „Které destinace navštívil uživatel s identifikátorem X?“. Vidíme, že v dotazu budou jistě figurovat dvě entity, konkrétně: *Uživatel* a *Destinace*. Při pohledu na grafové schéma je zřejmé, že abychom dostali všechny navštívené destinace pro konkrétního uživatele, musíme v grafu projít i přes všechny vrcholy s labelem *Cesta*, které uživatel podniknul. Tyto vrcholy spojují uživatele s destinacemi, které navštívil. Jedná se tedy o known walk, neboť víme, že musíme provést cesty obsahující vrcholy s labely *Uživatel*, *Cesta* a *Destinace*.

Nyní je otázkou, v kterých vrcholech je dobré průchod začít. Volba počátečních vrcholů má často velký vliv na výsledný čas výpočtu. Jednou z možností je začít na vrcholech s labelem *Destinace*. Grafový vzor by pak určoval cestu ze všech vrcholů s labelem *Destinace* přes hrany spojující tyto vrcholy s vrcholy s labelem *Cesta*. Následně pro všechny vrcholy s labelem *Cesta* zkontrolujeme, zda se jich vrchol s labelem *Uživatel*, který má shodný identifikátor, zúčastnil. Destinace, které by takto nebyly s uživatelem spojeny, by byly vyfiltrovány. Vyhovující destinace by byly vráceny. Ačkoliv by tento výpočet vedl ke správnému výsledku, procházet přes všechny destinace a tedy i přes všechny cesty není nejvhodnější. Vrcholů, které bychom museli zkontrolovat, je zbytečně mnoho. Obdobně nevhodný by bylo začínat ve vrcholech s labelem *Cesta*. Nejvýhodnějším řešením je začít ve vrcholu představujícím konkrétního uživatele. Následně projít přes vrcholy s labelem *Cesta*, kterých se uživatel zúčastnil a dostat se tak ke všem destinacím, které daný uživatel navštívil. Obecně je doporučeno začínat v co nejmenším počtu vrcholů. Výsledný dotaz v Cypheru ukazuje zdrojový kód 3. Do odpovědi je zahrnuta též informace, kolikrát daný uživatel destinace navštívil. Výsledky jsou podle toho seřazeny.

```
1 WITH 3 AS id
2 MATCH (u:User {userId:id})-[:PARTICIPATED]->(:Journey)-[:LED_TO]->(d
   :Destination)
3 RETURN d.destinationName, COUNT(d) AS count
4 ORDER BY count DESC, d.destinationName ASC;
```

Zdrojový kód 3: Known walk.

5 Hledání cest mezi 2 vrcholy

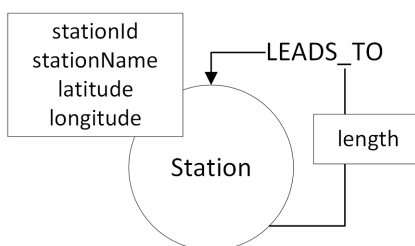
V této kapitole si odhalíme další velkou výhodu grafových databází a to hledání cest mezi vrcholy. Ne vždy platí, že cestu mezi dvěma vrcholy známe. Často máme pouze počáteční a cílový vrchol a chceme zjistit, zda je jeden z druhého dosažitelný, nebo hledáme nejkratší cestu mezi nimi. V první části si připomeneme dva základní algoritmy pro průchod grafem a následně této znalosti využijeme

ve druhé části při zjišťování dosažitelnosti a hledání nejkratších cest ve smyslu počtu hran mezi dvěma vrcholy. Ve třetí části se podíváme na hledání cesty s nejmenším možným součtem ohodnocení hran na této cestě.

Pro ukázkové příklady je použitý dataset⁹ obsahující informace o belgických železničních stanicích a spojeních mezi nimi. Jednoduchý model znázorňuje obrázek 9. V databázi tedy budeme uchovávat tedy pouze vrcholy s labellem Station a hrany mezi nimi s labellem LEADS_TO. Původní dataset je rozdělen do dvou csv souborů. Jeden uchovává informace o železničních stanicích a druhý spojení mezi nimi. Informace obsahující souřadnice pro jednotlivá spojení byly vypuštěny. Seznam souborů shrnuje tabulka 2.

Tabulka 2: Tabulka souborů pro ukázkou hledání cest.

Typ	Název souboru	Počet řádků	Informace o souboru
Entity	stations.csv	560	Informace o železničních stanicích.
Vztahy	leads_to.csv	1386	Vztah mezi stanicemi.



Obrázek 9: Model grafové databáze pro ukázkové příklady v této kapitole.

5.1 Procházení grafu

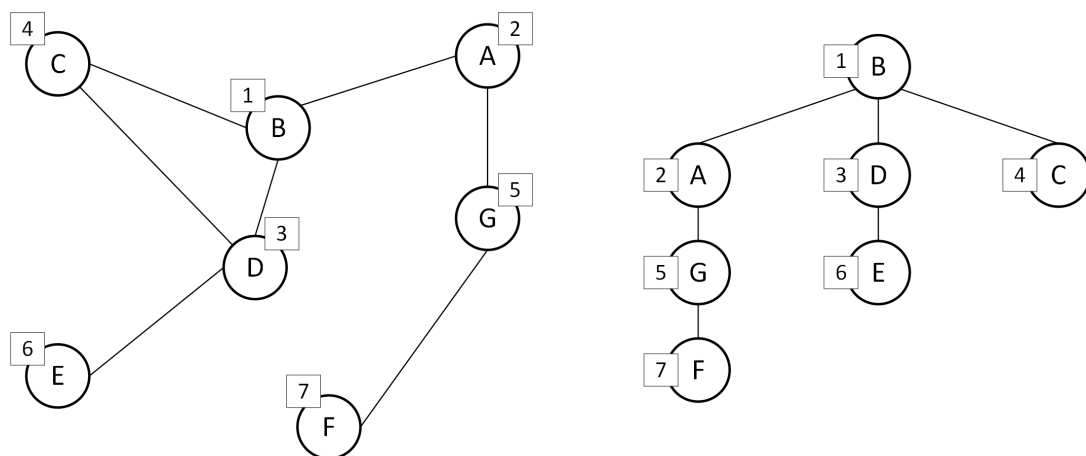
Pro prohledávání grafů od určitého počátečního vrcholu lze použít dva známé grafové algoritmy:

1. *Prohledávání do šířky*, označované jako BFS (z anglického Breadth-First Search)
2. *Prohledávání do hloubky*, označované jako DFS (z anglického Depth-First Search)

⁹https://opendata.infrabel.be/explore/dataset/station_to_station/information/?disjunctive.stationfrom_name&disjunctive.stationto_name&sort=stationfrom_name&basemap=jawg.light&location=8,50.51448,4.32709

Oba algoritmy mají složitost $O(|E| + |V|)$. Oba byly vymysleny pro hledání cesty v bludišti.

Algoritmus BFS začne v počátečním vrcholu a následně prohledá všechny své sousední vrcholy. Poté projde sousedy sousedů počátečního vrcholu, které ještě nenavštívil a tak postupuje dokud nenavštíví všechny dosažitelné vrcholy. Tento způsob procházení grafu lze znázornit pomocí kořenového stromu. Kořenem je počáteční uzel. Algoritmus prochází vrcholy grafu po jednotlivých úrovních tohoto stromu. Platí, že nejkratší cesta z počátečního vrcholu do vrcholu v v grafu je rovna hloubce v ve stromu, který vznikne průchodem do šířky. Lze jej efektivně implementovat pomocí fronty vrcholů. Průchod grafem do šířky a strom, vzniklý průchodem algoritmu jsou znázorněny na obrázku 10. Pořadí, v kterém jsou vrcholy procházeny je znázorněno čísly v obdélnících u každého vrcholu.

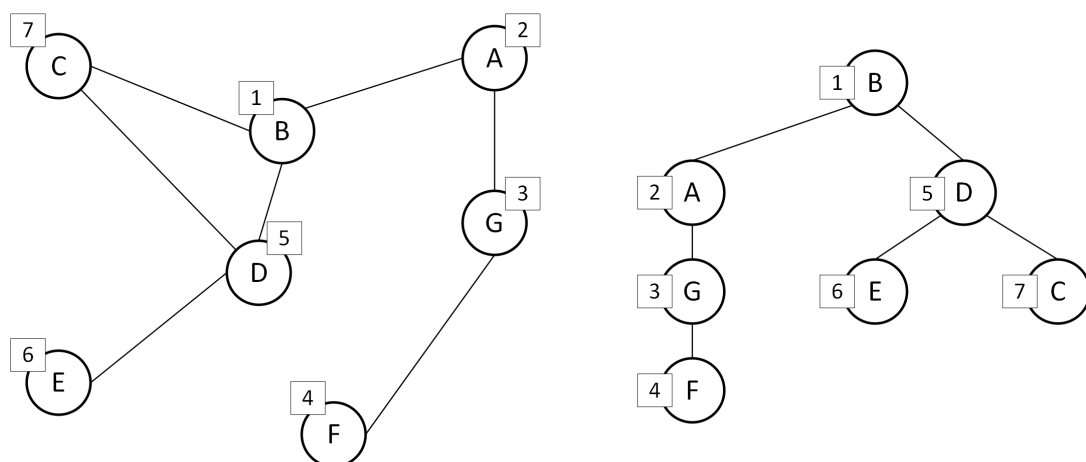


Obrázek 10: Průchod algoritmu BFS grafem.

Algoritmus DFS postupuje opačně, v aktuálně navštíveném vrcholu přejde do dosud nenavštíveného souseda, pokud takový soused existuje, jinak provede backtracking – vrací se zpět po cestě odkud přišel, dokud nenalezne vrchol jehož soused nebyl navštíven. Skončí jakmile jsou všechny vrcholy navštíveny. Tento způsob procházení grafu lze opět znázornit pomocí kořenového stromu, kde kořen je počáteční uzel. Avšak v tomto stromu pozorování ohledně nejkratších cest, jaké jsme udělali u BFS obecně neplatí. Při implementaci využíváme zásobník. Průchod grafem do hloubky a strom, vzniklý průchodem algoritmu jsou znázorněny na obrázku 11. Pořadí, v kterém jsou vrcholy procházeny je znázorněno čísly v obdélnících u každého vrcholu.

5.2 Cesta v neohodnoceném grafu

Na tento typ cesty lze nahlížet jako na cestu v grafu, kde každá hrana má stejnou váhu. Tyto cesty lze hledat i v grafu s ohodnocenými hranami tak, že ohodnocení hran ignorujeme. Počet hran na cestě pak určuje délku cesty. Avšak cest mezi



Obrázek 11: Průchod algoritmu DFS grafem.

dvěma vrcholy může být obrovské množství. Pokud neomezíme průchod grafem, například pomocí hledání cesty fixní délky, nebo maximální možným počtem hran na cestě, může se stát, že výpočet bude trvat velmi dlouho, nebo v horším případě bude nekonečný. Druhá možnost nastane pokud nezajistíme, že v grafu hledáme opravdu jen cesty a nikoliv sledy. Výpočet by se pak zacyklil a místo výsledku bychom dostali run-time-error.

Důvodů pro hledání cesty mezi dvěma vrcholy může být vícero. Jedním z nich je hledání cesty s nejkratší délkou. Tento problém bývá označován jako *SSSP*. Zkratka je odvozena z anglického *Single-Source Shortest Path*. Dalšími je zjištění dosažitelnosti, nebo hledání cesty o fixní délce.

Problém SSSP v neohodnoceném grafu lze vyřešit pomocí algoritmů na prohledávání grafů. Konkrétně se řeší pomocí prohledávání do šířky, neboť systematicky prochází všechny vrcholy, které mají vzdálenost od počátečního vrcholu rovnou k , kde $k \in \mathbb{N}_0$. Začíná pro $k = 0$ a postupně k zvětšuje o 1. V každém kroku kontroluje všechny vrcholy u , které mají z počátečního vrcholu vzdálenost rovnou k . Pokud narazí na cílový vrchol, vrací cestu délky k , kterou se do vrcholu dostal. S tímto přístupem je zřejmé, že pokud je cílový vrchol dosažitelný ze začínajícího vrcholu, pak v neohodnoceném grafu nalezneme nejkratší cestu. Nejkratších cest mezi dvěma vrcholy může být více. Pro libovolné $m \in \mathbb{N}$ můžeme takto postupovat i pro nalezení m různých nejkratších cest pro dva dané vstupní vrcholy.

V případě algoritmu procházení do hloubky bychom nalezení nejkratší cesty obecně dosáhnout nemuseli, i když by řešení mohlo být rychlejší, pokud by se algoritmus vydal správným směrem. To lze usoudit z toho, jak je algoritmus implementován.

V AQL lze cesty mezi stanicemi obsahující 1 až 4 hrany najít pomocí dotazu ve zdrojovém kódu 4. Na řádku 2 vyfiltrujeme ty sledy, které obsahují cyklus a specifikujeme strategii prohledávání do šířky. Výsledky jsou seřazeny podle

vzdálenosti. Tabulka 3 ukazuje výsledek tohoto dotazu po zavolání. Obrázek 12 pak znázorňuje podgraf, který obsahuje všechny vrcholy a hrany vystupující v dotazu. Tučnou čarou jsou zde vyznačeny nejkratší cesty mezi stanicemi.

```

1 FOR v,e,p IN 1..4 OUTBOUND 'Station/157' LEADS_TO
2 OPTIONS {uniqueVertices: 'path', order: 'bfs'}
3   FILTER p.vertices[-1]._id == 'Station/1061'
4   LET distance = LENGTH(p.edges[*])
5   LET stations = p.vertices[*].stationName
6   SORT distance
7   RETURN {
8     stations,
9     distance
10  }
```

Zdrojový kód 4: Cesty v neohodnoceném grafu.

Tabulka 3: Výsledná tabulka pro dotaz ve zdrojovém kódu 4.

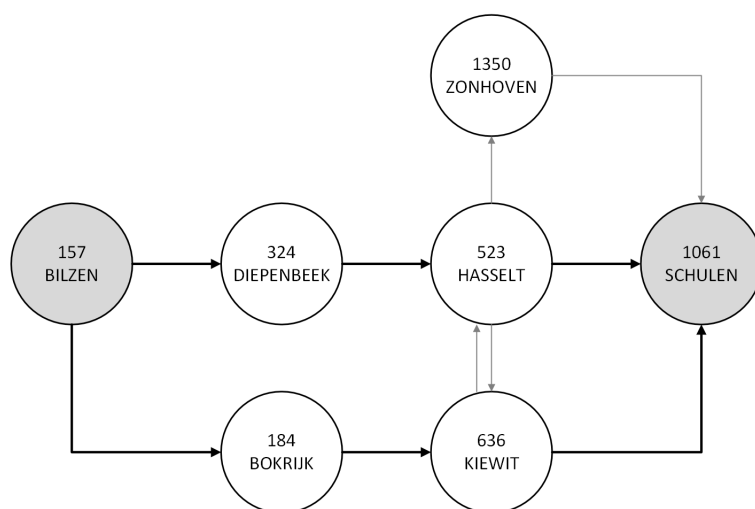
stations	distance
BILZEN, BOKRIJK, KIEWIT, SCHULEN	3
BILZEN, DIEPENBEEK, HASSELT, SCHULEN	3
BILZEN, BOKRIJK, KIEWIT, HASSELT, SCHULEN	4
BILZEN, DIEPENBEEK, HASSELT, KIEWIT, SCHULEN	4
BILZEN, DIEPENBEEK, HASSELT, ZONHOVEN, SCHULEN	4

5.3 Cesta v ohodnoceném grafu

Délka cesty mezi počátečním a koncovým vrcholem v ohodnoceném grafu je dána součtem ohodnocení hran na cestě. Tedy nejmenší počet hran na cestě nutně neznamená, že součet ohodnocení hran je menší než na jiné cestě obsahující více hran. Z tohoto důvodu zde nemůžeme pro vyřešení SSSP použít lehce modifikovaný algoritmus, který nám řešil problém v předchozí části.

Jedním ze způsobů, jak vyřešit SSSP, je udržování a aktualizování nejkratších nalezených cest do již navštívených vrcholů. Tento princip je známý z významného algoritmu pro řešení SSSP – Dijkstrově algoritmu. Je ho využito ve zdrojovém kódu 5 v jazyku Gremlin. Výsledek, který dostaneme shrnuje tabulka 4. Celková vzdálenost v je zaokrouhlená na celé metry.

Z vrácených výsledků vidíme, že nejkratší délka železnic mezi stanicemi SCHAEER-BEEK a MECHELEN má délku 18.2 kilometrů. Na této cestě projdeme přes 6 stanic. Jen o 50 metrů je delší druhá cesta, která obsahuje o jednu zastávku méně. Třetí výsledek je delší o více než 1.5 kilometru a jedná se o přímý spoj.



Obrázek 12: Podgraf reprezentující výsledek dotazu ve zdrojovém kódu 4.

Tabulka 4: Výsledná tabulka pro dotaz ve zdrojovém kódu 5.

stations	stationsCount	totalLength
SCHAERBEEK, BUDA, VILVOORDE, EPPEGEM, WEERDE, MECHELEN	6	18 206 m
SCHAERBEEK, VILVOORDE, EPPEGEM, WEERDE, MECHELEN	5	18 269 m
SCHAERBEEK, MECHELEN	2	19 833 m

Zde též platí, že cest může být mezi dvěma vrcholy opravdu mnoho. Obzvláště ve velikých grafech. Proto se často z časových důvodů nejkratší cesta aproximuje. Pokud má vrchol na možné cestě výstupní stupeň větší než nějaký limit l (například $\deg_{out}(u) > 10000$), cestu přes něj nepočítáme, neboť bychom museli procházet každou výstupní hranu což by bylo výpočetně náročné. Těmto vrcholům se říká super-uzly. Více o nich si řekneme v následující kapitole.

Výpočet lze omezit zadáním horního limitu pro délku cesty. Pokud délka cesty tento limit přesáhne, tak tato cesta je zavrhnuta. Limit můžeme určit tak, že spočítáme nejkratší cestu v grafu při zanedbání ohodnocení hran. Následně ohodnocení hran na této cestě sečteme.

Dijkstrův algoritmus nefunguje pro SSSP v grafu, kde se mohou vyskytovat hrany se záporným ohodnocením. V těchto grafech je proto nutné použít Bellman-Fordův algoritmus. Pro hledání nejkratších cest mezi všemi dvojicemi vrcholů (All-Pairs Shortest Paths – APSP) grafu je vhodný Floyd-Warshallův algoritmus.

Grafové databáze nabízí optimalizované algoritmy pro hledání nejkratší cesty v grafech. Na ty a na další se podíváme v následující kapitole.

```

1 g.withSack(0.0).V().
2   has("Station", "stationId", 1048).
3   repeat(outE("LEADS_TO").
4     sack(sum).by("length").
5     inV().as("visited").
6     simplePath().
7     group("minDist").
8     by().by(sack().min()).
9     filter(project("currMinDist", "sackDist").
10       by(select("minDist").select(select("visited"))).
11       by(sack()).
12       where("currMinDist", eq("sackDist")))).
13   until(has("Station", "stationId", 810)).
14   order().
15     by(sack(), asc).
16   limit(3).
17   project("stations", "stationsCount", "totalLength").
18     by(path().
19       unfold().
20       hasLabel("Station").
21       values("stationName").
22       fold()).
23     by(path().count(local).math('(_+1)/2')).
24     by(sack()).
25   fold()

```

Zdrojový kód 5: Hledání nejkratší cesty v ohodnoceném grafu.

6 Algoritmy

Tato kapitola byla napsána pomocí knih [21] a [18]. Dále byla využita dokumentace knihovny Graph Data Science Library [12] a zdroje [22] a [3].

V této kapitole si ukážeme řadu užitečných algoritmů, které jsou určeny pro práci s grafem a jsou tedy v určitých situacích vhodné aplikovat při dotazech na grafovou databázi. Umožňují nám efektivně hledat nejkratší cesty mezi vrcholy, nacházet vrcholy, které pro nás mohou být svým způsobem důležité, nebo třeba analyzovat komunity (skupinky propojených vrcholů) v grafu. Tyto algoritmy a mnohé další obsahuje knihovna s názvem *Graph Data Science Library*. Tuto knihovnu lze dodatečně nainstalovat do databáze Neo4j. Zkráceně ji budeme označovat *GDS*.

Pro nejefektivnější běh algoritmů používá *GDS* k reprezentaci dat grafu speciální formát grafu uložený dočasně v paměti. Jedná se o strukturu vrcholů a hran, jejichž vlastnosti mohou být pouze číselné. Každý takový graf je pojmenován a díky tomu na něj můžeme odkazovat v algoritmech. Tyto odkazy jsou uloženy v *grafovém katalogu*. Graf lze vytvořit více způsoby. V práci je použitý způsob vytvoření grafu pomocí *Cypher projekce*.

Algoritmy pak mohou běžet ve třech režimech:

- *stream* – výsledky algoritmu obdržíme jako výsledek dotazu.
- *mutate* – výstup algoritmu aktualizuje graf v paměti.
- *write* – zapíše výsledek algoritmu zpět do databáze Neo4j.

6.1 Algoritmy pro hledání cesty v grafu

V praxi je užitečné v grafu najít rychle optimální cestu mezi dvěma, či více vrcholy. Tento problém potřebují řešit například nástroje pracující s grafy reprezentující mapy, které hledají vhodné trasy mezi body na mapě. Dalšími příklady optimalizačních problémů jsou řízení dodavatelských řetězců, hledání nejvhodnějšího autobusového spojení a analýza počítačové sítě.

Pro řešení tohoto problému vznikly různé algoritmy. Každý pracuje odlišně a je výhodný pro různé poddruhy problémů. Pár z nich si zde představíme.

6.1.1 Dijkstra

Jeden z nejznámějších algoritmů pro hledání nejkratší cesty. Lze jej aplikovat na neohodnocené grafy, nebo grafy s kladným ohodnocením hran. Jedná se o hladový algoritmus, který k specifikovanému vrcholu vydá nejkratší cestu z tohoto vrcholu do všech ostatních vrcholů. Lze jej také omezit, aby iteroval jen dokud nenajde nejkratší cestu ke konkrétnímu cílovému vrcholu. Správné řešení je dosaženo díky pamatování si dosud nejkratších nalezených cest do již navštívených vrcholů. V každé iteraci pak vybere jeden vrchol s nejkratší nalezenou cestou ze všech, které dosud nebyly vybrány a následně aktualizuje všechny nejkratší cesty vedoucí přes tento vybraný vrchol. Pokud je nějaký vrchol nedosažitelný nebo ještě nebyl navštívený, má hodnotu ∞ .

Nevýhodou tohoto algoritmu je, že pro nalezení nejkratší cesty pro 2 zadané vrcholy je v nejhorším případě potřeba nalézt všechny nejkratší cesty do ostatních vrcholů, než se algoritmus dostane k požadovanému výsledku. To nastává v případě, kdy mezi dvěma vrcholy je nejkratší cesta nejdelší ze všech nejkratších cest z počátečního vrcholu do ostatních vrcholů.

6.1.2 A*

A* pracuje podobně jako Dijkstrův algoritmus. Avšak často dosahuje výsledku rychleji díky dodatečné informaci, kterou dostane na vstupu. Této informaci se říká heuristika. Je to funkce $h: V \rightarrow \mathbb{R}^+$ odhadující délku cesty ze všech vrcholů do cílového vrcholu.

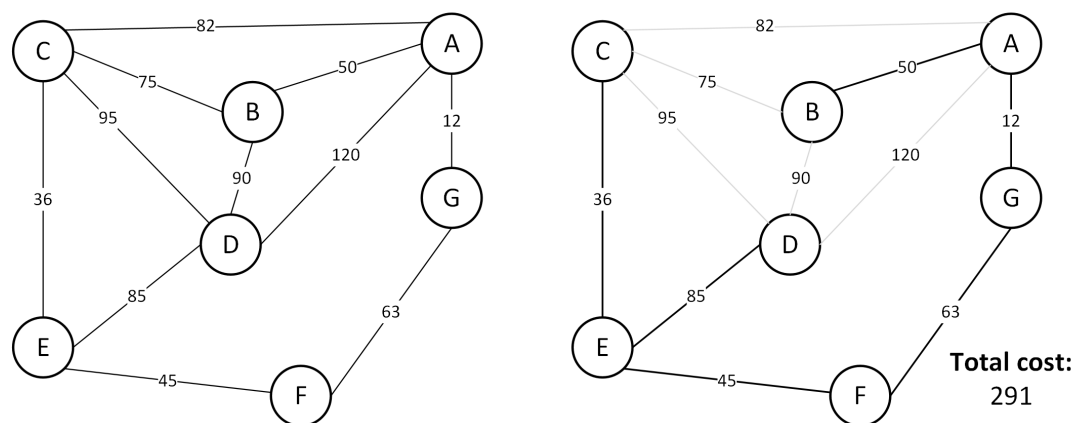
Rozhodnutí do jakého vrcholu bude algoritmus pokračovat je provedeno na základě funkce $f(n) = g(n) + h(n)$ pro každý navštívený vrchol n . Funkce $g(n)$ představuje součet ohodnocení nejkratší cesty ze startu přes dosud navštívené vrcholy do n . Funkce $h(n)$ je vstupní heuristika. V každé iteraci pak algoritmus pokračuje vrcholem, pro který je hodnota funkce f nejmenší.

Pro správné výsledky by heuristika měla být optimistická, tedy odhad by měl být menší nebo roven skutečné vzdálenosti do cíle. Platí, že čím lépe odhadneme heuristiku tím rychleji algoritmus najde řešení. Dijkstraův algoritmus je speciálním případem A^* . Pro něj platí $h(v) = 0$ pro všechny vrcholy $v \in V$.

V *GDS* lze A^* použít například pro vrcholy reprezentující nějakou lokaci, která je přesně určena pomocí zeměpisné šířky a délky. Geografické souřadnice nám slouží jako část heuristické funkce. Pro svůj rychlý výpočet je například implementován ve videohrách.

6.1.3 Minimální kostra grafu

Dalším, mezi informatiky dobře známým, problémem je hledání minimální kostry v souvislém grafu. Vstupem je souvislý neorientovaný graf G s ohodnocenými hranami. Výstupem je minimální kostra grafu G . Kostru grafu lze definovat jako podgraf grafu G , který je stromem obsahujícím všechny vrcholy původního grafu. My požadujeme, aby kostra byla minimální, tedy aby součet ohodnocení hran v nalezeném podgrafu byl ze všech možných koster nejmenší. Příklad minimální kostry grafu je na obrázku 13.



Obrázek 13: Minimální kostra grafu.

Problém kostry grafu má více variant. Můžeme též hledat maximální kostru grafu, nebo omezit počet vrcholů v kostře na $k \in \mathbb{N}$ tomuto problému se říká k -spanning tree.

GDS algoritmus na hledání kostry obsahuje. Konkrétně nabízí Jarníkův-Primův algoritmus, poprvé popsáný v roce 1930 českým matematikem Vojtěchem Jarníkem. Tento iterativní hladový algoritmus vyžaduje na vstupu specifikovat kromě grafu i počáteční vrchol s . Postupně je z něj vytvářen strom T tak, že v každém kroku je k T připojena jedna hrana. Vždy je vybrána hrana s nejmenší vahou, která vede z T do vrcholu mimo T . Pokud neexistuje hrana, která by šla do stromu přidat, výpočet skončí. T je minimální kostra komponenty vstupního

grafu obsahující s . Pokud při implementaci využíváme binární prioritní frontu máme celkovou složitost $O(|E| \log_2 |V|)$.

Tento algoritmus lze využít pro nejlevnější propojení měst vodním potrubím, kde nižší ohodnocení hran znamená menší náklady. Dalším využitím je aproximace problémů patřících do třídy NPTIME, jako je například problém obchodního cestujícího.

6.1.4 Random walk

Random walk provádí náhodné sledy zadané délky. Simuluje průchod grafem přičemž náhodně vybírá hrany. V klasické verzi algoritmu je v každém vrcholu pravděpodobnost vybrání hrany uniformní a nezáleží na předchozích výběrech. *GDS* nabízí Random walk druhého řádu. U tohoto algoritmu se specifikují dva parametry:

- `returnFactor` – vyjadřuje tendenci algoritmu se vracet zpět k naposledy navštívenému vrcholu. Hodnoty menší než 1 pravděpodobnost tohoto výběru zvyšují.
- `inOutFactor` – vyjadřuje tendenci algoritmu zůstat či nezůstat v blízkosti počátečního vrcholu. Čím vyšší hodnota je zadána, tím bude průchod grafem lokálnější.

V klasické verzi jsou oba parametry nastaveny na hodnotu 1. Pravděpodobnost výběru též můžeme ovlivnit pomocí ohodnocených hran. Vlastnost hrany, která je vyšší než jedna pravděpodobnost výběru této hrany zvyšuje. Naopak hodnoty z intervalu $(0, 1)$ pravděpodobnost zmenšují.

Random walk se používá jako součást jiných algoritmů, kde je potřeba množina náhodných, ale navzájem propojených vrcholů. V následující ukázce byl algoritmus použit na železniční stanice 9. Algoritmus začínal ve stanici s identifikátorem 9. Tabulka 5 ukazuje výsledky algoritmu. Pro jedno nastavení parametrů jsou ukázány tři náhodné cesty. Cesta je reprezentována ve druhém sloupci řádkem identifikátorů vrcholů oddělených středníkem, přes které prochází. V třetím sloupci je pro každou cestu spočítána vzdálenost vzdušnou čarou z počátečního do cílového vrcholu.

Tabulka 5: Random walk.

Parametry	Vrcholy na cestě	Vzdálenost
returnFactor = 20; inOutFactor = 0,2	9; 132; 187; 541; 814; 142; 732; 644	29 596 m
	9; 686; 1134; 1270; 326; 1061; 523; 19	38 943 m
	9; 1238; 553; 1174; 368; 648; 916; 1260	19 693 m
returnFactor = 0,2; inOutFactor = 20	9; 132; 9; 132; 9; 132; 686; 132	8 181 m
	9; 686; 9; 686; 9; 686; 9; 686	9 587 m
	9; 132; 9; 686; 9; 686; 9; 132	8 181 m

6.2 Algoritmy pro míru centrality

V rámci grafu můžeme identifikovat důležitost vrcholů pomocí algoritmů centrality. Můžeme detekovat jedince, které jsou svým způsobem v grafu významější než ostatní. Tyto algoritmy se dají využít například k hledání kritických částí v telekomunikačních nebo počítačových sítích, tedy těch přes které proudí spousta informací a jejichž výpadek by představoval velké potíže. Hledání důležitých lidí v rámci organizace. Hledání odlehlých dat, které mohou odhalit, že dochází k podvodům a podobně. *GDS* nabízí spoustu algoritmů pro měření centrality. My se podíváme na pár z nich.

6.2.1 Degree Centrality

Jedná se o nejznámější a nejjednodušší míru centrality. Každému vrcholu v síti přiřazuje skóre podle stupně daného vrcholu. V sociálních sítích mívají vrcholy i stupně v řádu desítek milionů (jedná se o vrcholy, reprezentující účty známých osobností nebo organizací). Těmto vrcholům říkáme super-uzly. Abychom prošli všechny sousedy super-uzlu potřebujeme značně více času, než v případě „obyčejného“ vrcholu. To se negativně projevuje v časové odezvě u dotazů pracujících s takovými vrcholy. Degree centrality nám umožňuje takové vrcholy odhalit. Vrcholy s větším stupněm také lze obecně považovat za důležitější, neboť přes ně může proudit více informací díky velkému počtu hran. Příkladem může být graf reprezentující železniční síť. Čím více máme z dané železniční stanice spojení, tím spíše se jedná o významější stanici.

6.2.2 Closeness Centrality

Nejvlivnější vrchol je ten, pro který je součet nejkratších cest do všech ostatních vrcholů co nejmenší. Algoritmy pro Closeness a Betweenness Centrality mohou být v případě velkých grafů výpočetně dost náročné. Proto se často přistupuje k jiným, ne zcela přesným, zato mnohonásobně rychlejším, algoritmům.

Skóre vrcholu $u \in V$ v grafu $G = \langle V, E \rangle$ se pak počítá podle vzorce

$$closenessCentrality(u) = \frac{1}{\sum_{v \in V} dist(u, v)}, \quad (1)$$

nebo se normalizuje

$$normalizedClosenessCentrality(u) = \frac{|V| - 1}{\sum_{v \in V} dist(u, v)}. \quad (2)$$

V případě, že graf není souvislý, lze použít *harmonic centrality*, které se vyřadí s nedosažitelnými vrcholy.

6.2.3 Betweenness Centrality

Algoritmus spočítá nejkratší cesty mezi všemi možnými páry vrcholů. Následně každý vrchol obdrží skóre, které je tím vyšší, čím více nejkratších cest vede přes

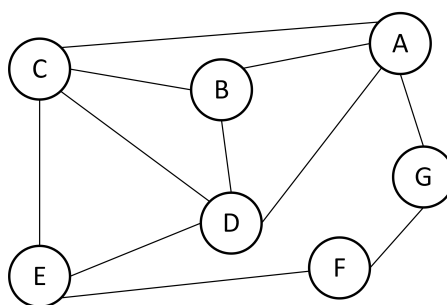
daný vrchol. Tyto hodnoty mohou pro nás být klíčové například v grafu, kde vrcholy představují křižovatky a hrany cesty mezi nimi. Informaci, kterými křižovatkami prochází nejvíce nejkratších cest, můžeme využít pro odhad lokací, kde se pravděpodobně bude pohybovat nejvíce lidí a tedy je potřeba zvýšit bezpečnost.

6.2.4 PageRank

S tímto algoritmem přišla na trh společnost Google. Byl určena pro hodnocení významnosti webových stránek. Webové stránky si lze představit jako vrcholy grafu. Hrany mezi nimi reprezentují odkazy na stránkách odkazující na jiné stránky. Důležitost vrcholu je určena nejen důležitostí vrcholu samotného, ale také důležitostí vrcholů na tuto stránku odkazujících. Vysoké skóre vrcholu tedy znamená, že je vrchol propojen s vrcholy, které mají též vysoké skóre. Toto řešení je založeno na vlastních vektorech a stochastických maticích z lineární algebry. Výpočet skóre většinou vede na nekonečný výpočet a proto se omezuje počet iterací tohoto výpočtu, nebo se udává maximální možná odchylka. PageRank se používá na orientované grafy. Pro neorientované grafy lze použít algoritmus eigenvector centrality.

6.2.5 Použití algoritmů pro míru centrality

Výsledky aplikace zmíněných algoritmů pro měření centrality na graf z obrázku 14 je znázorněn v tabulce 6.



Obrázek 14: Příklad grafu, na kterém jsou prováděny algoritmy.

Tabulka 6: Výsledky měření centrality.

Vrchol	A	B	C	D	E	F	G
Degree Centrality	4	3	4	4	3	2	2
Closeness Centrality	0,75	0,6	0,75	0,75	0,67	0,55	0,6
Betweenness Centrality	3,33	0	1,33	1,33	2,67	1	1,33
Eigen Vector Centrality	0,45	0,41	0,49	0,49	0,32	0,14	0,17

6.3 Detekce komunit

Tyto algoritmy lze využít pro hledání určitých skupin vrcholů, které mají mezi sebou více hran, než mají s ostatními vrcholy grafu. Těmto skupinám říkáme komunity. Příklad lze nalézt v sociálních sítích, kde můžeme sledovat komunity blízkých přátel, které tvoří shluky propojených vrcholů. Znalost komunit lze též využít pro doporučování produktů uživatelům s podobným nákupním chováním.

Algoritmy pro detekci komunit se dělí na *deterministické* a *nedeterministické*. Deterministické vrací pro stejný graf vždy stejný výsledek, zatímco výsledek u nedeterministických může být pro stejný graf mírně odlišný. Mezi deterministické se řadí počítání trojúhelníků, výpočet shlukovacího koeficientu, nebo hledání komponent. Mezi nedeterministické patří například Label Propagation, nebo Louvain Modularity.

6.3.1 Počítání trojúhelníků

Algoritmus pro každý vrchol určí počet *trojúhelníků*, kterých je daný vrchol součástí. Trojúhelník je zde definován jako skupina tří navzájem propojených vrcholů. Trojúhelníky nám pomáhají určit do jaké míry jsou vrcholy v našem grafu propojeny. V hustých grafech, lze obvykle nalézt více trojúhelníků. Algoritmus je oblíbený při analýze sociálních sítí, kde pomáhá detekovat komunity, nebo zkoumání skupin webových stránek s podobným námětem podle odkazů mezi nimi.

Počet trojúhelníků lze využít též pro výpočet *shlukovacího koeficientu* vrcholu, který určuje pravděpodobnost, s jakou jsou jeho sousedé propojeni (tvoří s ním trojúhelník). Shlukovací koeficient vrcholu u je pak určen vzorcem

$$\text{ClusteringCoefficient}(u) = \frac{2 * \text{TriangleCount}(u)}{\text{deg}(u) * (\text{deg}(u) - 1)}. \quad (3)$$

Pro ukázkový graf na obrázku 14 počet trojúhelníků a shlukovací koeficient jednotlivých vrcholů zobrazuje tabulka 7. Vidíme, že vrchol A je součástí tří trojúhelníků, které jsou tvořeny vrcholy A-B-C, A-B-D a A-C-D. Stupeň vrcholu je 4. Dosazením do vzorce 3 dostáváme shlukovací koeficient 0.5.

Tabulka 7: Počet trojúhelníků a shlukovací koeficient.

Vrchol	A	B	C	D	E	F	G
Triangle Count	3	3	4	4	1	0	0
Clustering Coefficient	0,5	1	0,67	0,67	0,33	0	0

6.3.2 Hledání komponent

Nalezení komponent v grafu je užitečné pro pochopení našich dat. Algoritmy pro tuto úlohu jsou rychlé. Mohou být využívány v grafech, které se často aktualizují,

pro detekci nových vrcholů mezi skupinami. Toho lze pak využít při různých analýzách. Existují algoritmy pro hledání silně a slabě souvislých komponent.

Pro graf na obrázku 14 nám algoritmus pro hledání slabě souvislých komponent vrátí, že všechny vrcholy grafu jsou součástí jedné komponenty.

6.3.3 Algoritmus Louvain

Algoritmus se snaží maximalizovat modularitu komunit. Modularita je způsob jakým můžeme měřit kvalitu komunit. Porovnává komunity v grafu s náhodným grafem, který vznikne odstraněním hran v původním grafu a následným přidáním těchto hran mezi dva náhodně zvolené nepropojené vrcholy. Předpokládá se rovnoměrné spojení těchto vrcholů. Hodnota je vždy mezi -1 a 1. Čím je hodnota vyšší tím jsou vrcholy v komunitách propojenější. Výpočet modularity je počítán vzorcem

$$M = \frac{1}{2m} \sum_{u,v} [A_{uv} - \frac{k_u k_v}{2m}] \delta(C_u, C_v), \quad (4)$$

kde

- u a v jsou vrcholy,
- m je součet přes všechna ohodnocení hran,
- A_{uv} je váha hrany mezi vrcholy u a v ,
- k_x je součet přes všechna ohodnocení hran incidentních s x
- a $\delta(C_u, C_v) = \begin{cases} 1 & \text{jestliže } u \text{ i } v \text{ patří do stejné komunity,} \\ 0 & \text{jinak.} \end{cases}$

Podíl $\frac{k_u k_v}{2m}$ ve vzorci 4 odpovídá náhodnému grafu mající stejnou distribuci stupňů a tedy vyjadřuje očekávanou sumu ohodnocení hran mezi dvěma vrcholy.

Na začátku každý vrchol tvoří komunitu. V neohodnoceném grafu má každá hrana váhu 1. Algoritmus má dva kroky:

1. Každý vrchol v grafu je přiřazen do nějaké komunity. Následně dochází k vyhodnocování změny modularity, pokud vrchol přeřadíme do jiné komunity. Vyhodnocuje se pomocí vzorce 4. Cílem je nejlepší rozřazení jednotlivých vrcholů do komunit.
2. Každá komunita tvoří vrchol nového grafu. Ohodnocené hrany v novém grafu představují jak jsou komunity mezi sebou propojeny. Hrana má takovou váhu, kolik mezi komunitami existuje spojení. Tento nový graf bude vstupem v další iteraci.

Algoritmus iteruje tak dlouho, dokud dochází ke zvyšování modularity ve vzniklých komunitách.

Pro ukázkový graf 14 nám algoritmus rozdělil vrcholy do dvou komunit. Výsledky shrnuje tabulka 8.

Tabulka 8: Algoritmus Louvain.

Vrchol	A	B	C	D	E	F	G
Komunita	2	2	2	2	6	6	6

7 Experimenty

Zde budou experimenty se zvolenými databázemi.

Závěr

Závěr práce v „českém“ jazyce. V budoucnu je možné přidat entitu cestovní kanceláře, která bude pořádat cesty do destinací a uživatelé po objednání budou do této cesty připojeni. To otevírá dveře dalším entitám například pro hodnocení cestovních kanceláří, nebo vztahům které se týká zaměstnanosti uživatelů u dané cestovky.

Conclusions

Thesis conclusions in “English”.

Literatura

- [1] Wilson, Robin James. *Introduction to graph theory*. 4th Edition. 1996. ISBN 0-582-24993-7.
- [2] Ramba, Jaroslav. *Úvod do grafové terminologie / Ramba.cz*. 2024. Dostupný také z: <https://www.ramba.cz/blog/grafova-terminologie-a-dostupne-technologie>.
- [3] Bělohávek, Radim. 2024. Dostupný také z: <http://belohlavek.inf.upol.cz/vyuka/DiskretniStruktury-2020.pdf>.
- [4] 2024. Dostupný také z: <https://home.zcu.cz/~ryjacek/students/DMA/skripta/8.pdf>.
- [5] Robinson, Ian; Webber, Jim; Eifrem, Emil. *Graph databases*. 2nd Edition. 2015. ISBN 978-1-491-93200-1.
- [6] *A Guide to Graph Databases*. 2024. Dostupný také z: <https://www.influxdata.com/graph-database/>.
- [7] McCreary, Dan. 2024. Dostupný také z: <https://dmccreary.medium.com/how-to-explain-index-free-adjacency-to-your-manager-1a8e68ec664a>.
- [8] *Série Znalostní grafy: Díl 2: Datový model RDF*. 2024. Dostupný také z: <https://data.gov.cz/%C4%8Dl%C3%A1nky/znalostn%C3%AD-grafy-02-rdf>.
- [9] Štráfelda, Jan. *Knowledge Graph*. 2024. Dostupný také z: <https://www.strafelda.cz/knowledge-graph>.
- [10] 2024. Dostupný také z: http://langer.zam.slu.cz/teaching/dbaii/olap_oltp.pdf.
- [11] IONOS. *IONOS Digital Guide*. 2024. Dostupný také z: <https://www.ionos.com/digitalguide/hosting/technical-matters/graph-database/>.
- [12] *Neo4j Documentation*. 2024. Dostupný také z: <https://neo4j.com/docs>.
- [13] *Neo4j Developer Center*. 2024. Dostupný také z: <https://neo4j.com/developer>.
- [14] *OrientDB Documentation*. 2024. Dostupný také z: <https://www.orientdb.com/docs/latest/index.html>.
- [15] *ArangoDB Documentation*. 2024. Dostupný také z: <https://docs.arango-db.com>.
- [16] Lawrence, Kelvin R. *Practical gremlin: An apache tinkerpops tutorial*. 2024. Dostupný také z: <https://kelvinlawrence.net/book/PracticalGremlin.html>.
- [17] *TinkerPop Documentation*. 2024. Dostupný také z: <https://tinkerpop.apache.org/docs>.

- [18] Bechberger, Dave; Perryman, Josh. *Graph databases in action: Examples in Gremlin*. 1st Edition. 2020. ISBN 978-1-61729-637-6.
- [19] Broecheler, Matthias; Gosnell, Denise Koessler. *Practitioner's Guide to graph data: Applying graph thinking and graph technologies to solve complex problems*. 1st Edition. 2020. ISBN 978-1-492-04407-9.
- [20] Macrometa. *Macrometa*. 2024. Dostupný také z: [⟨https://www.macrometa.com/docs/graphs/graph-concepts/graphs-and-indexes⟩](https://www.macrometa.com/docs/graphs/graph-concepts/graphs-and-indexes).
- [21] Needham, Mark; Hodler, Amy E. *Graph Algorithms: Practical Examples in Apache Spark and Neo4j*. 1st Edition. 2019. ISBN 978-1-492-05781-9.
- [22] 2024. Dostupný také z: [⟨https://diana.ms.mff.cuni.cz/pogamut_files/lectures/2015-2016/04.2-Pathfinding-Bajer060410.pdf⟩](https://diana.ms.mff.cuni.cz/pogamut_files/lectures/2015-2016/04.2-Pathfinding-Bajer060410.pdf).